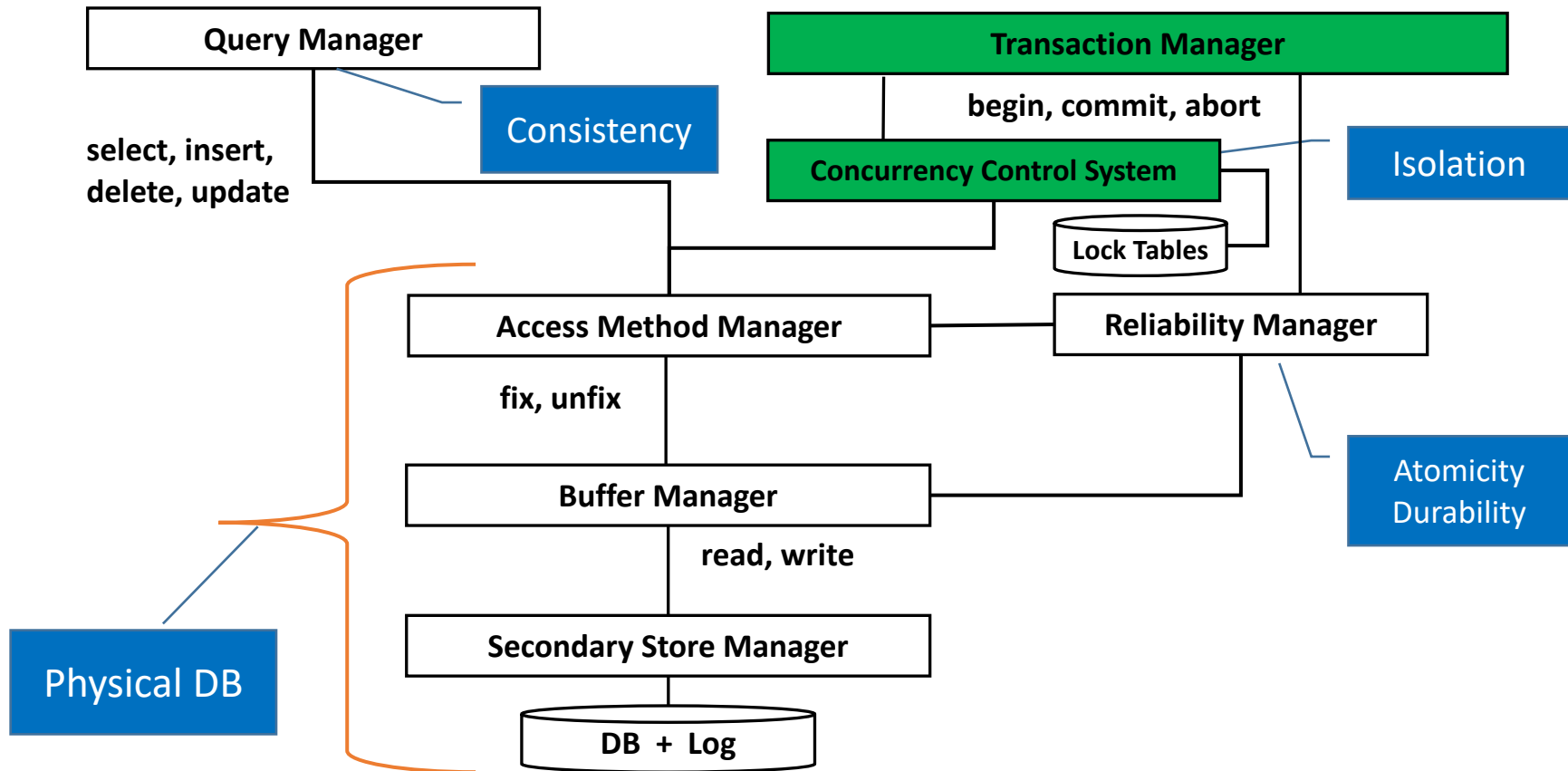


Data bases 2

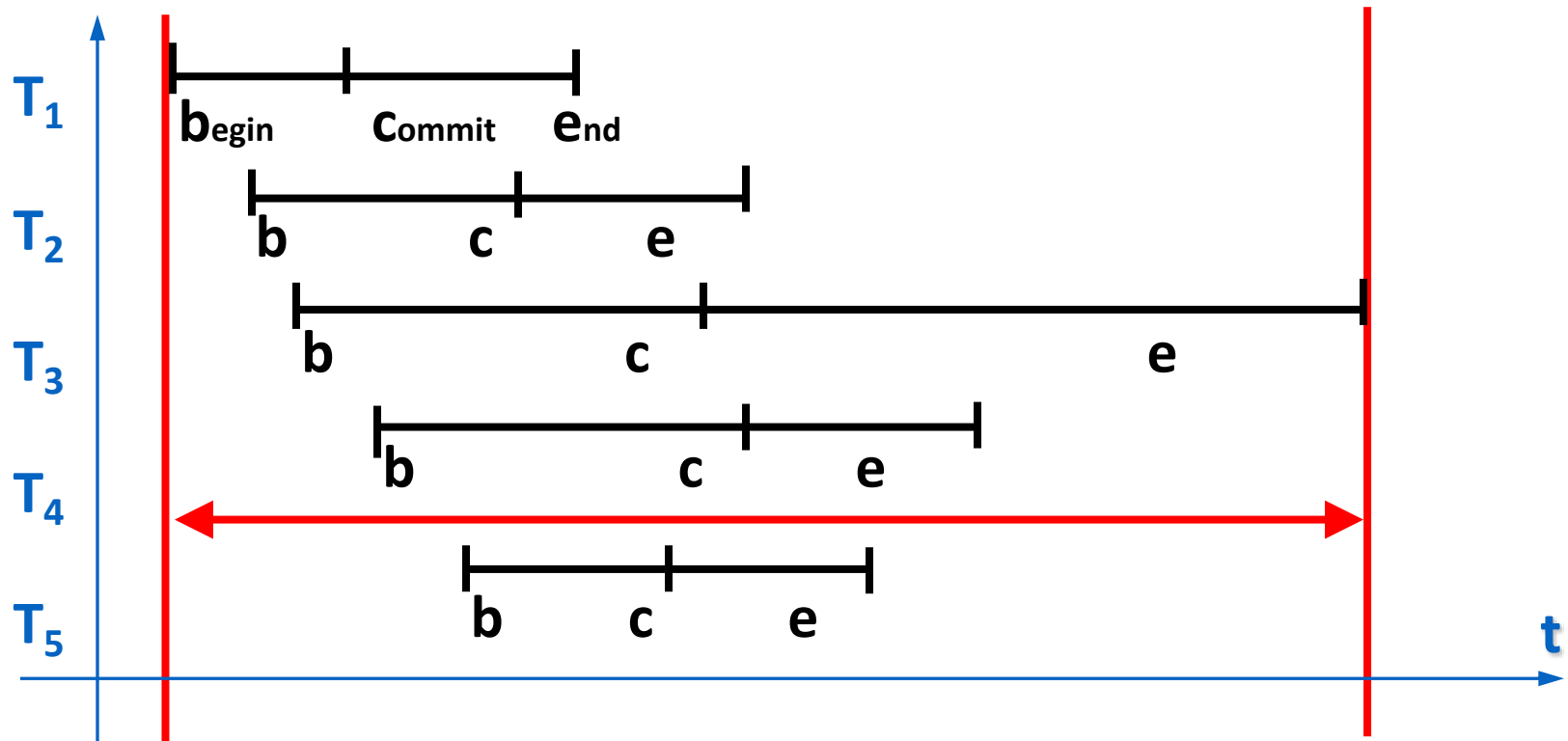
Concurrency Control

Concurrency in the DBMS architecture



- The Concurrency Control System
 - Manages the simultaneous execution of transactions
 - Avoids the insurgence of anomalies
 - While ensuring performance

Advantages of Concurrency



- Goal: exploit parallelism to maximise transactions per second (TPS)

Problems due to Concurrency

T_1 : **begin transaction**

update account

set balance = balance + 3

where customer = 'Smith'

commit work

T_2 : **begin transaction**

update account

set balance = balance + 6

where customer = 'Smith'

commit work

Concurrent SQL transactional statements addressing the same resource

T_1 : **begin transaction**

D = D + 3

commit work

T_2 : **begin transaction**

D = D + 6

commit work

T_1 : **begin transaction**

var x;

read(D → x)

x = x + 3

write(x → D)

commit work

T_2 : **begin transaction**

var y;

read(D → y)

y = y + 6

write(y → D)

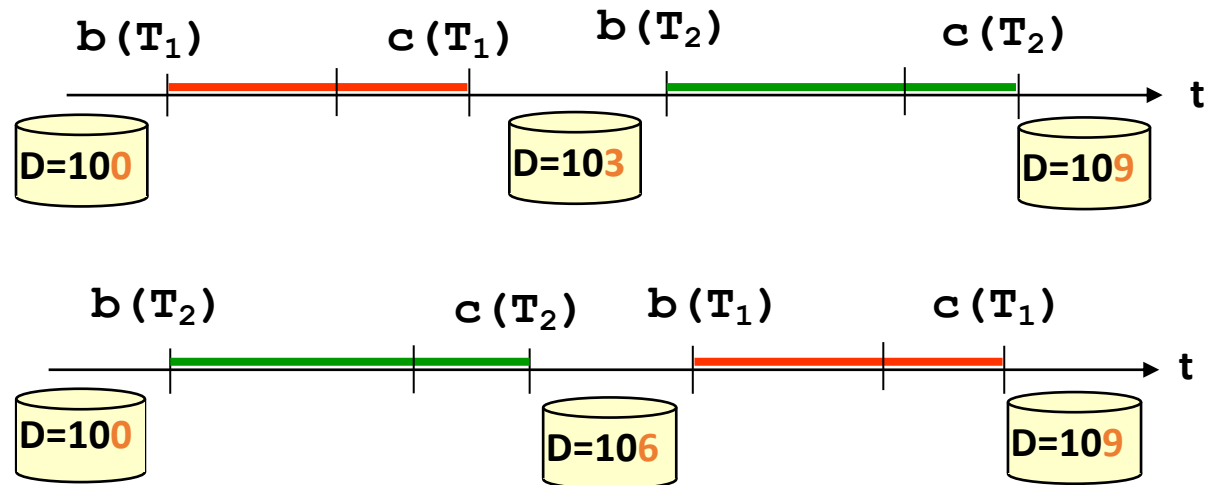
commit work

Serial Executions

```
bot(T1)  
  var x;  
  r(D → x)  
  x = x + 3  
  w(x → D)  
commit(T1)
```

```
bot(T2)  
  var y;  
  r(D → y)  
  y = y + 6  
  w(y → D)  
commit(T2)
```

Initially D₀=100

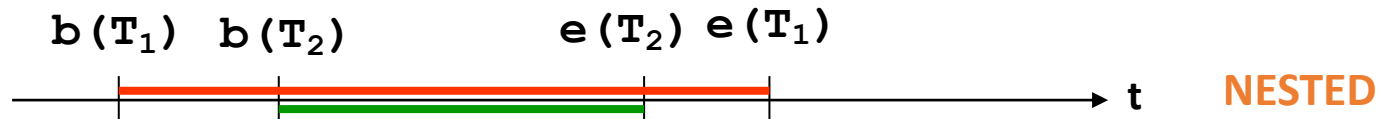
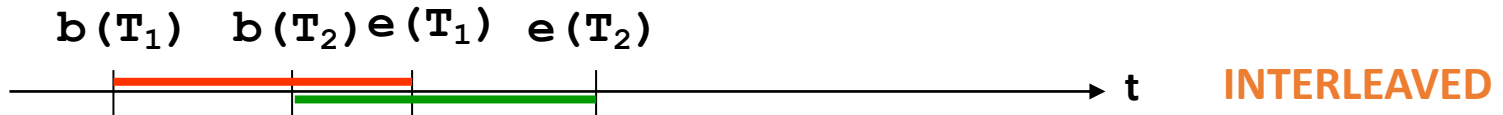
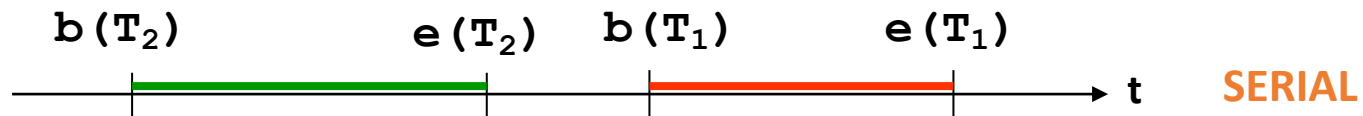
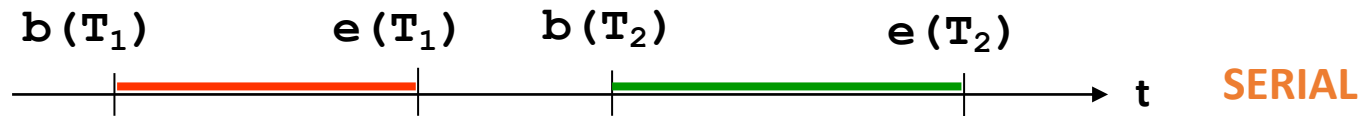


Note: it is not important that the final value be the same

Concurrency Control

- Concurrency is fundamental
 - Tens or hundreds of transactions per second cannot be executed serially
- Examples: banks, ticket reservations
- Problem: concurrent execution may cause anomalies
 - Concurrency needs to be controlled

Concurrent Executions



Execution with Lost Update

D = 100

1 $T_1: r(D \rightarrow x)$

2 $T_1: x = x + 3$

3 $T_2: r(D \rightarrow y)$

4 $T_2: y = y + 6$

5 $T_1: w(x \rightarrow D)$

6 $T_2: w(y \rightarrow D)$

D = 103

D = **106 !**

T_1 : UPDATE account

SET balance = balance + 3

WHERE client = 'Smith'

T_2 : UPDATE account

SET balance = balance + 6

WHERE client = 'Smith'

- Note: this anomaly does not depend merely on T_2 overwriting the value produced by T_1
 - $w_1(D), w_2(D)$ is ok (serial)
 - $r_1(D), w_1(D), w_2(D)$ is ok too (serial)
 - $r_1(D), r_2(D), w_1(D), w_2(D)$ is not ok: inconsistent updates **from the same initial value**

Sequence of I/O actions producing the error



or



Dirty Read

T_1 : **UPDATE** account
 SET balance = balance + 3
 WHERE client = 'Smith'

T_2 : **UPDATE** account
 SET balance = balance + 6
 WHERE client = 'Smith'

$D = 100$

1 T_1 : $r(D \rightarrow x)$

2 T_1 : $x = x + 3$

3 T_1 : $w(x \rightarrow D)$ $D = 103$

4 T_2 : $r(D \rightarrow y)$ this read is “dirty” (uncommitted value)

5 T_1 : **rollback**

6 T_2 : $y = y + 6$

7 T_2 : $w(y \rightarrow D)$ $D = 109 !$

Non-repeatable Read

D = 100

1 T₁: r(D → x)

2 T₂: r(D → y)

3 T₂: y = y + 6

4 T₂: w(y → D) D = 106

5 T₁: r(D → z) z <> x !

T₁ : **SELECT** balance **FROM** account
 WHERE client = 'Smith'

...

SELECT balance **FROM** account
 WHERE client = 'Smith'

T₂ : **UPDATE** account
 SET balance = balance + 6
 WHERE client = 'Smith'

Phantom Update

Constraint: $A+B+C=100$, $A=50$, $B=30$, $C=20$

$T_1: r(A \rightarrow x), r(B \rightarrow y) \dots$

$T_2: r(B \rightarrow s), r(C \rightarrow t)$

$T_2: s = s + 10, t = t - 10$

$T_2: w(s \rightarrow B), w(t \rightarrow C)$ (now $B=40$, $C=10$, $A+B+C=100$)

$T_1: r(C \rightarrow z)$ (but, for T_1 , $x+y+z = A+B+C = 90!$)

- So for T_1 it is as if “somebody else” had updated the value of the sum
- But for T_2 the update is perfectly legal (does not change the value of the sum)

Phantom Insert

Tab	
A	B

T_1 : $C = \text{AVG}(B: A=1)$ $C = \text{select avg}(B) \text{ from Tab where } A=1$

T_2 : **Insert (A=1,B=2)** $\text{insert into Tab values (1, 2)}$

T_1 : $C = \text{AVG}(B: A=1)$ $C = \dots$

- Note: this anomaly does not depend on data already present in the DB when T_1 executes, but on a “phantom” tuple that is inserted by “someone else” and satisfies the conditions of a previous query of T_1

Summary of Anomalies

- Lost update $r_1 - r_2 - w_2 - w_1$
 - An update is applied from a state that ignores a preceding update, which is lost
- Dirty read $r_1 - w_1 - r_2 - \text{abort}_1 - w_2$
 - An uncommitted value is used to update the data
- Non-repeatable read $r_1 - r_2 - w_2 - r_1$
 - Someone else updates a previously read value
- Phantom update $r_1 - r_2 - w_2 - r_1$
 - Someone else updates data that contributes to a previously valid constraint
- Phantom insert $r_1 - w_2(\text{new data}) - r_1$
 - Someone else inserts data that contributes to a previously read datum

Concurrency Theory vs System Implementation

- **Model:** an abstraction of a system, object or process, which purposely disregards details to simplify the investigation of relevant properties
- Concurrency Theory builds upon a model of transactions and concurrency control principles that helps understanding real systems
- Real systems exploit implementation level mechanisms (locks, snapshots) which help achieve some of the desirable properties postulated by the theory
 - Do not look for a view or conflict serializability checker in your DBMS
 - Look instead for lock tables, lock types, lock granting rules, snapshots, etc..
 - Understand how the implementation mechanisms ensure properties modelled by the Concurrency Theory

Transactions, Operations, Schedules

- Operation: a read or write of a specific datum by a specific transaction
 - NB: the schedule notation omits the program variables
 - $r_1(x) = r_1(x \rightarrow .)$
 - $w_1(x) = w_1(. \rightarrow x)$
 - $r_1(x)$ and $r_1(y)$ are different operations
 - $r_1(x)$ and $r_2(x)$ are different operations
 - $w_1(x)$ and $w_2(x)$ are different operations
- Schedule: a sequence of operations performed by concurrent transactions that respects the order of operations of each transaction
 - T_1 : $r_1(x) w_1(x)$
 - T_2 : $r_2(z) w_2(z)$
 - S_1 : $r_1(x) r_2(z) w_1(x) w_2(z)$

Schedules

- How many distinct schedules exist for two transactions?

$$N = 6$$

- With T_1 and T_2 from the previous slide:

- $r_1(x) \ w_1(x) \ r_2(z) \ w_2(z)$

- $r_2(z) \ w_2(z) \ r_1(x) \ w_1(x)$

- $r_1(x) \ r_2(z) \ w_1(x) \ w_2(z)$

- $r_2(z) \ r_1(x) \ w_2(z) \ w_1(x)$

- $r_1(x) \ r_2(z) \ w_2(z) \ w_1(x)$

- $r_2(z) \ r_1(x) \ w_1(x) \ w_2(z)$

} *serial*

} *interleaved*

} *nested*

Schedules: how many?

- How many serial schedules exist for n transactions? N_S
- How many distinct schedules exist for n transactions? N_D
- n transactions $(T_1, T_2, \dots, T_i, \dots, T_n)$, each with k_i operations
- T_1 has k_1 operations, T_i has k_i operations...

$T_1: o_{11} \ o_{12} \ \dots \ o_{1 \ k_1}$

$T_2: o_{21} \ o_{22} \ \dots \ o_{2 \ k_2}$

...

$T_i: o_{i1} \ o_{i2} \ \dots \ o_{i \ k_i}$

...

$T_n: o_{n1} \ o_{n2} \ \dots \ o_{n \ k_n}$

$$N_S = n!$$

$$N_D = \frac{(\sum_{i=1}^n k_i)!}{\prod_{i=1}^n (k_i!)}$$

$$N_S \ll N_D$$

- N_S = number of permutations of N transactions
- N_D = number of permutations of all operations /
product of the number of permutations of the operations of
each transaction (only 1 out of the $k_i!$ permutations is valid,
the one that respects the sequence of operations)

Principles of Concurrency Control

- Goal: to reject schedules that cause anomalies
- **Scheduler**: a component that accepts or rejects the operations requested by the transactions
- **Serial schedule**: a schedule in which the actions of each transaction occur in a contiguous sequence
- S: $r_0(x) r_0(y) w_0(x) r_1(y) r_1(x) w_1(y) r_2(x) r_2(y) r_2(z)$
 $w_2(z)$

Principles of Concurrency Control

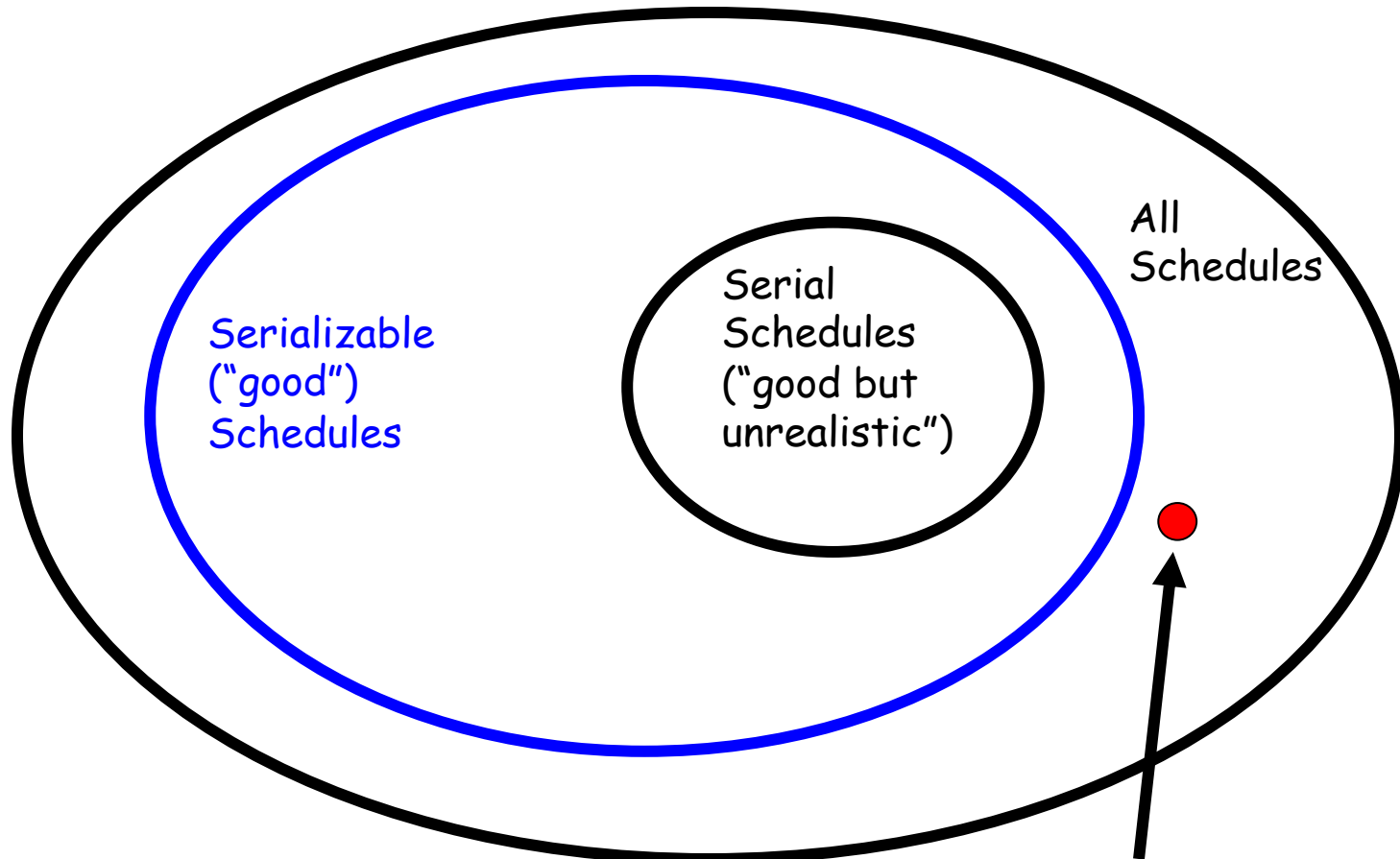
- **Serializable schedule**

- A schedule that leaves the database in the same state as **some** serial schedule of the same transactions
- Commonly accepted as a notion of schedule **correctness**
- Requires a notion of schedule equivalence to identify classes of schedules that ensure serializability
 - Different notions → different classes (cost of checking if schedule is serializable)

- **Assumption**

- We initially assume that transactions are observed “a-posteriori” and limited to those that have committed (**commit-projection**), and we decide whether the observed schedule is admissible
- In practice (and in contrast), schedulers must make decisions **while transactions are running**

Basic Idea



This schedule **may** generate anomalies.
Its execution leads to a database state that
no serial execution would produce

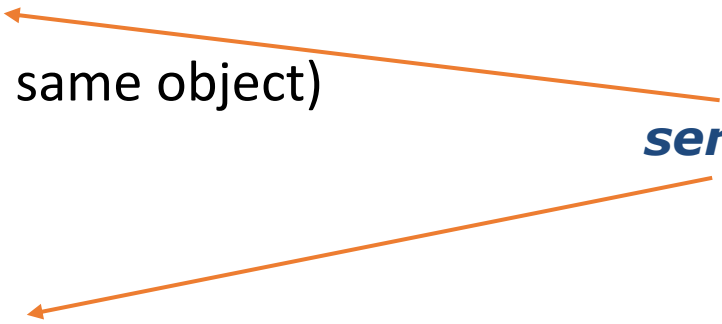
View-serializability

- Preliminary definitions:
 - $r_i(x)$ **reads-from** $w_j(x)$ in a schedule S when $w_j(x)$ precedes $r_i(x)$ and there is no $w_k(x)$ in S between $r_i(x)$ and $w_j(x)$
 - $w_i(x)$ in a schedule S is a **final write** if it is the last write on x that occurs in S
- Two schedules are **view-equivalent** ($S_i \approx_v S_j$) if: they have 1) the same operations, 2) the same reads-from relationship, and 3) the same final writes
- A schedule is **view-serializable** if it is view-equivalent to a serial schedule of the same transactions
- The class of view-serializable schedules is named **VSR**
- **Mnemonically:** S is view-serializable if
 - 1) every read operation sees the same values and
 - 2) the final value of each object is written by the same transaction
- as if the transactions were executed serially in **some** order

Note

- The reads-from relationship **$r_i(x)$ reads-from $w_j(x)$** in a schedule S assumes that $r_i(x)$ reads the value written by $w_j(x)$ independently of the time at which the commit of T_j occurs
 - In other words the value written by $w_j(x)$ could be uncommitted when $r_i(x)$ reads it but we are sure (by definition of commit projection) that it will be committed

Examples of view-serializability

- S_3 : $w_0(x)$ $r_2(x)$ $r_1(x)$ $w_2(x)$ $w_2(z)$
 - S_4 : $w_0(x)$ $r_1(x)$ $r_2(x)$ $w_2(x)$ $w_2(z)$
 - (switch of read operations on the same object)
 - S_5 : $w_0(x)$ $r_1(x)$ $w_1(x)$ $r_2(x)$ $w_1(z)$
 - S_6 : $w_0(x)$ $r_1(x)$ $w_1(x)$ $w_1(z)$ $r_2(x)$
 - (switch of read/write ops on different objects that preserves final write)
 - S_3 is view-equivalent to serial schedule S_4 (so it is view-serializable)
 - S_5 is not view-equivalent to S_4 (different operations) but is view-equivalent to serial schedule S_6 , so it is also view-serializable
- serial***
- 

Examples of view-serializability

- $S_7 : r_1(x) \ r_2(x) \ w_1(x) \ w_2(x)$
- $S_8 : r_1(x) \ r_2(x) \ w_2(x) \ r_1(x)$
- $S_9 : r_1(x) \ r_1(y) \ r_2(z) \ r_2(y) \ w_2(y) \ w_2(z) \ r_1(z)$
- S_7 corresponds to a lost update
- S_8 corresponds to a non-repeatable read
- S_9 corresponds to a phantom update
- They are all **non** view-serializable (check by testing view equivalence with T_1, T_2 and with T_2, T_1)

A More Complex Example

$S_{10} : w_0(x) \ r_1(x) \ w_0(z) \ r_1(z) \ \mathbf{r_2(x)} \ w_0(y) \ r_3(z) \ w_3(z) \ w_2(y) \ \mathbf{w_1(x)} \ w_3(y)$

- Is S_{10} view-serializable?
 - Yes iff there exists a serial schedule S_s s.t. $S_{10} \approx_v S_s$
- Let's try with $S_{11} : T_0 \ T_1 \ T_2 \ T_3$ is **not** view-equivalent to S_{10}
- $w_0(x) \ w_0(z) \ w_0(y) \ r_1(x) \ r_1(z) \ \mathbf{w_1(x)} \ \mathbf{r_2(x)} \ w_2(y) \ r_3(z) \ w_3(z) \ w_3(y)$
- Let's try with $S_{12} : T_0 \ T_2 \ T_1 \ T_3$
- $w_0(x) \ w_0(z) \ w_0(y) \ r_2(x) \ w_2(y) \ r_1(x) \ r_1(z) \ w_1(x) \ r_3(z) \ w_3(z) \ w_3(y)$

A More Complex Example

S_{10} : $w_0(x)$ $r_1(x)$ $w_0(z)$ $r_1(z)$ $r_2(x)$ $w_0(y)$ $r_3(z)$ $w_3(z)$ $w_2(y)$ $w_1(x)$ $w_3(y)$

S_{12} : $w_0(x)$ $w_0(z)$ $w_0(y)$ $r_2(x)$ $w_2(y)$ $r_1(x)$ $r_1(z)$ $w_1(x)$ $r_3(z)$ $w_3(z)$ $w_3(y)$

A More Complex Example

S_{10} : $w_0(x)$ $r_1(x)$ $w_0(z)$ $r_1(z)$ $r_2(x)$ $w_0(y)$ $r_3(z)$ $w_3(z)$ $w_2(y)$ $w_1(x)$ $w_3(y)$

S_{12} : $w_0(x)$ $w_0(z)$ $w_0(y)$ $r_2(x)$ $w_2(y)$ $r_1(x)$ $r_1(z)$ $w_1(x)$ $r_3(z)$ $w_3(z)$ $w_3(y)$

reads-from OK: $r_1(x)$ from $w_0(x)$,

$r_1(z)$ from $w_0(z)$,

$r_2(x)$ from $w_0(x)$,

$r_3(z)$ from $w_0(z)$,

final writes OK: $w_1(x)$, $w_3(y)$, $w_3(z)$

$S_{10} \in \text{VSR}$



A More Complex Example

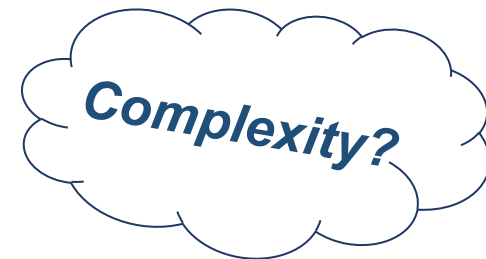
S_{10} : $w_0(x)$ $r_1(x)$ $w_0(z)$ $r_1(z)$ $r_2(x)$ $w_0(y)$ $r_3(z)$ $w_3(z)$ $w_2(y)$ $w_1(x)$ $w_3(y)$

S_{13} : $w_0(x)$ $w_0(z)$ $w_0(y)$ $r_2(x)$ $w_2(y)$ $r_3(z)$ $w_3(z)$ $w_3(y)$ $r_1(x)$ $r_1(z)$ $w_1(x)$

reads-from : $r_1(x)$ from $w_0(x)$,
 $r_1(z)$ from $w_3(z)$

S_{10} not view-equivalent to T_0, T_2, T_3, T_1

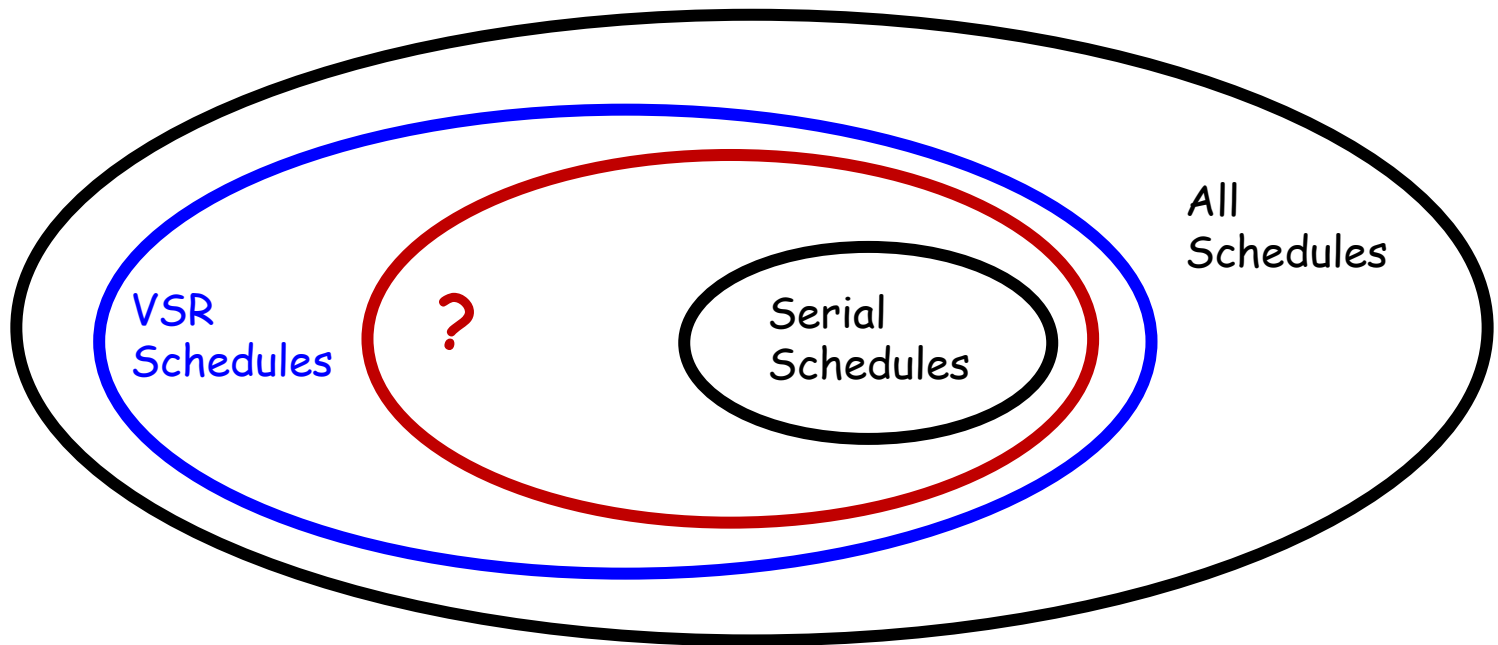
Different reads-from relationship



Complexity of view-serializability

- Deciding view-equivalence of two given schedules is done in polynomial time and space
- Deciding if a **generic** schedule is in VSR is an **NP-complete problem**
 - requires considering the reads-from and final writes of all possible serial schedules with the same operations – combinatorial in the general case
 - Performance!! what can we trade for that?
 - ...Accuracy!
- We look for a **stricter definition that is easier to check**
 - May lead to **rejecting some schedule that would be acceptable under view-serializability** but not under the stricter/faster criterion

There are “too many” VSR schedules



Conflict-serializability

- Preliminary definition:
 - Two operations o_i and o_j ($i \neq j$) are in **conflict** if they address the same resource and at least one of them is a write
 - **read-write** conflicts (r-w or w-r)
 - **write-write** conflicts (w-w)

Conflict-serializability

- Two schedules are **conflict-equivalent** ($S_i \approx_c S_j$) if :
 - S_i and S_j contain the same operations and in all the conflicting pairs the transactions occur in the same order
- A schedule is **conflict-serializable** iff it is conflict-equivalent to a serial schedule of the same transactions
- The class of conflict-serializable schedules is named **CSR**

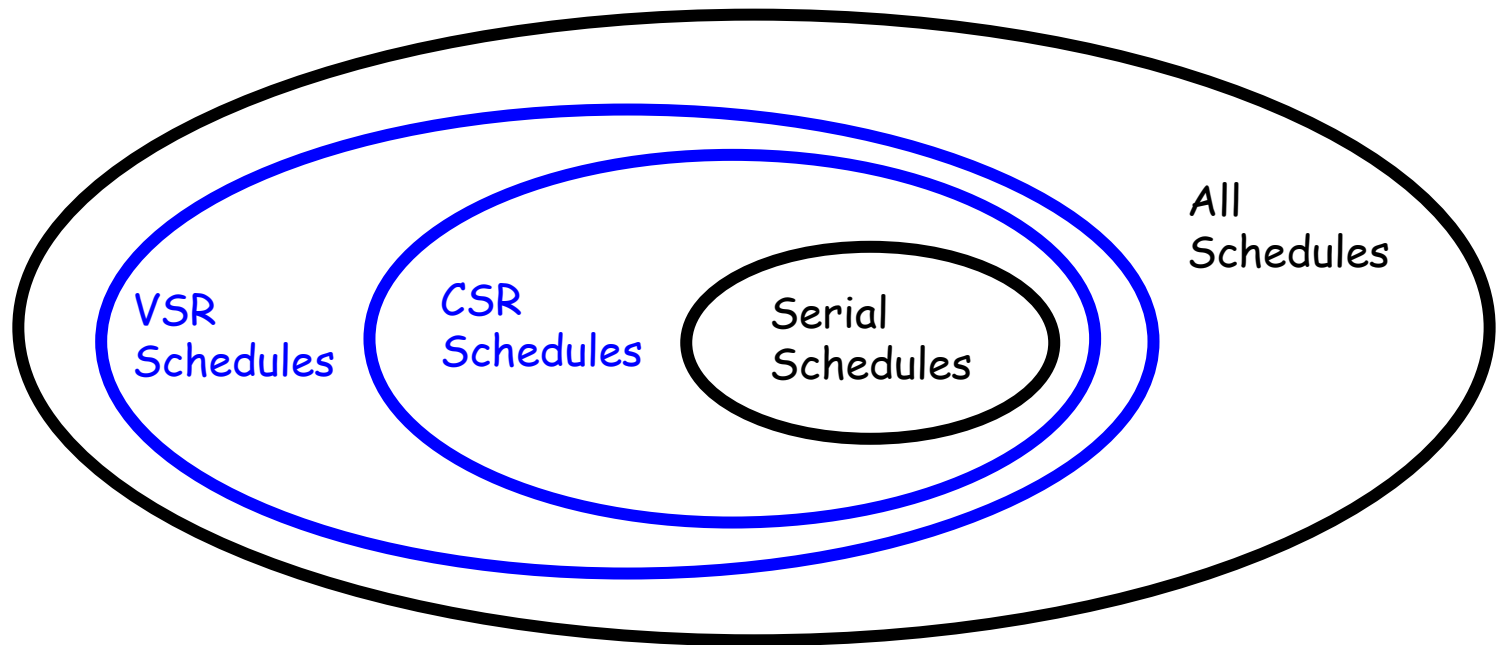
Relationship between CSR and VSR

- **VSR $\supset$ CSR** : all conflict-serializable schedules are also view-serializable, but the converse is not necessarily true
- Proof: there are VSR schedules not in CSR
- Counter-example: we consider $r_1(x) w_2(x) w_1(x) w_3(x)$ that
 - is view-serializable: it is view-equivalent to
$$T_1 T_2 T_3 = r_1(x) w_1(x) w_2(x) w_3(x)$$
 - is not conflict-serializable, due to the presence of
$$r_1(x) w_2(x) \text{ and } w_2(x) w_1(x)$$
 - there is no conflict-equivalent serial schedule
 - neither $T_1 T_2 T_3$ w.r.t. which it is view equivalent
 - nor $T_2 T_1 T_3$ which preserves the final write on x
 - nor all the others..

CSR implies VSR

- **CSR $\rightarrow$ VSR**: conflict-equivalence $\approx_c$ implies view-equivalence $\approx_v$
- We assume $S1 \approx_c S2$ and prove that $S1 \approx_v S2$
- $S1$ and $S2$ must have:
 - The same final writes: if they didn't, there would be at least two writes in a different order, and since two writes are conflicting operations, the schedules would not be $\approx_c$
 - The same “reads-from” relations: if not, there would be at least one pair of conflicting operations in a different order, and therefore, again, $\approx_c$ would be violated

CSR and VSR



Testing conflict-serializability

- Is done with a *conflict graph* that has:
 - One node for each transaction T_i
 - One arc from T_i to T_j if there exists at least one conflict between an operation o_i of T_i and an operation o_j of T_j such that o_i precedes o_j
- Theorem:
 - A schedule is in CSR if and only if its conflict graph is acyclic

Testing conflict-serializability

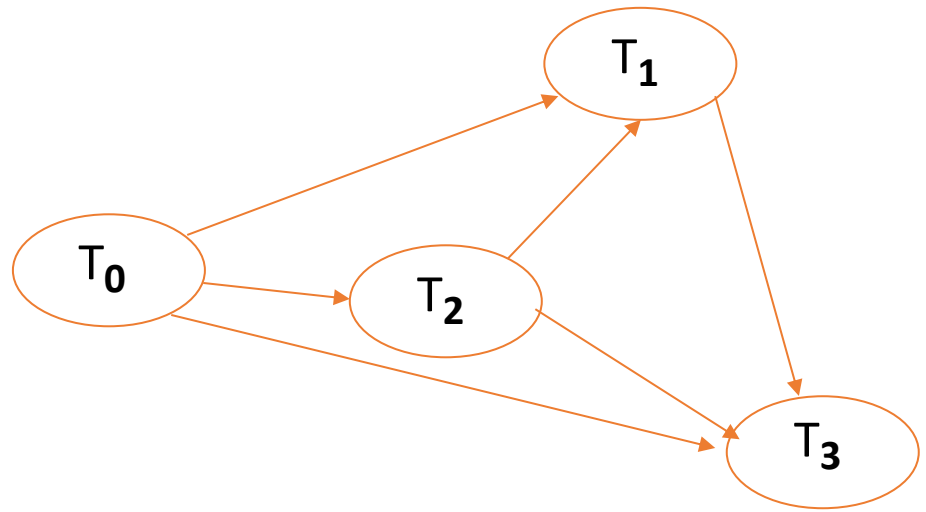
$S_{10} : w_0(x) \ r_1(x) \ w_0(z) \ r_1(z) \ r_2(x) \ w_0(y) \ r_3(z) \ w_3(z) \ w_2(y) \ w_1(x) \ w_3(y)$

- Resource-based projections:

- $\mathbf{x} : w_0 \ r_1 \ r_2 \ w_1$

- $\mathbf{y} : w_0 \ w_2 \ w_3$

- $\mathbf{z} : w_0 \ r_1 \ r_3 \ w_3$

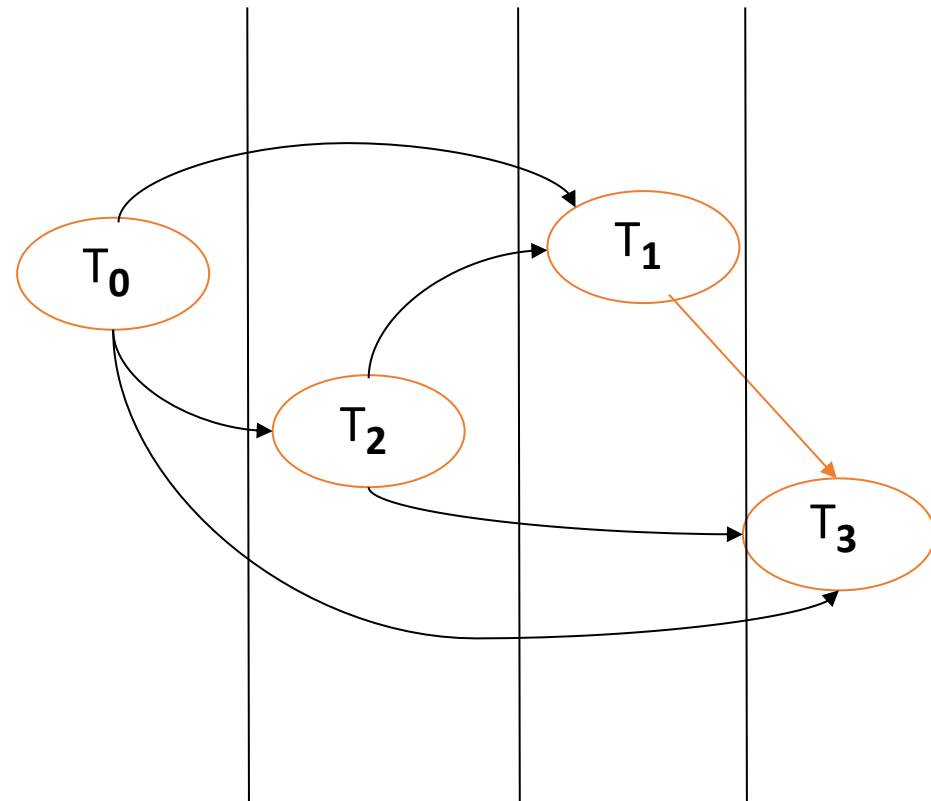


CSR implies acyclicity of the CG

- Consider a schedule S in CSR. As such, it is $\approx_c$ to a serial schedule
- W.l.o.g. we can (re)label the transactions of S to say that their order in the serial schedule is: $T_1 T_2 \dots T_n$
- Since the serial schedule has all conflicting pairs in the same order as schedule S , in the conflict graph there can only be arcs (i,j) , with $i < j$
- Then the graph is acyclic, as a cycle requires at least an arc (i,j) with $i > j$

Acyclicity of the CG implies CSR

- If S 's graph is acyclic then it induces a topological (partial) ordering on its nodes, i.e., an ordering such that the graph only contains arcs (i,j) with $i < j$. The same partial order exists on the transactions of S
- Any serial schedule whose transactions are ordered according to the partial order is conflict-equivalent to S , because for all conflicting pairs (i,j) it is always $i < j$
 - In the example before: $T_0 < T_2 < T_1 < T_3$
 - In general, there can be many compatible serial schedules (i.e., many serializations for the same acyclic graph)
 - As many as the total orders compatible with the partial topological order



Let's go back...

- $r_1(x) w_2(x) w_1(x) w_3(x)$
 - it is VSR – it is view-equivalent to
- $T_1 T_2 T_3 = r_1(x) w_1(x) w_2(x) w_3(x)$
 - it is not CSR, **due to the presence of the cycle between $r_1(x) w_2(x)$ and $w_2(x) w_1(x)$**
 - there is no conflict-equivalent serial schedule, neither $T_1 T_2 T_3$ nor $T_2 T_1 T_3$
 - Try to understand the relationship between CSR and VSR...

Concurrency Control in Practice

- CSR checking would be efficient if we knew the graph from the beginning — but we don't
- A scheduler must rather work “**online**”, i.e., decide for each requested operation whether to execute it immediately or to reject/delay it
- It is not feasible to maintain the conflict graph, update it, and check its acyclicity at each operation request
- The assumption that concurrency control can work only with the commit-projection of the schedule is unrealistic, **aborts do occur**
- Some simple on-line “decision criterion” is required for the scheduler, which must
 - **avoid as many anomalies as possible**
 - **have negligible overhead**

Arrival sequences vs a posteriori schedules

- So far the notation
 - $r_1(x) w_2(x) w_1(x) w_3(x)$
- represented a “schedule”, which is an **a posteriori view** of the execution of concurrent transactions in the DBMS (also called “history” in some books)
- A schedule represents “what has happened”, “which operations have been executed by which transaction in which order”
- They can be further restricted by the **commit-projection hypothesis** to operations executed by **committed** transactions
- When dealing with “online” concurrency control, it is important also to consider **“arrival sequences”**, i.e., sequences of operation requests emitted in order by transactions
- With an abuse of notation, we will denote an arrival sequence in the same way as a posteriori schedule
 - $r_1(x) w_2(x) w_1(x) w_3(x)$
- The distinction will be clear from the context
- The CC system maps an arrival sequence into an effective a posteriori schedule

Concurrency control approaches

- How can concurrency control be implemented “online”?
- Two main families of techniques:
 - Pessimistic
 - Based on locks, i.e., resource access control
 - If a resource is taken, make the requester wait or pre-empt the holder
 - Optimistic
 - Based on timestamps and versions
 - Serve as many requests as possible, possibly using out-of-date versions of the data
- We will compare the two families after introducing their features
- Commercial systems take the best of both worlds

Locking

- It's the most common method in commercial systems
- A transaction is well-formed w.r.t. locking if
 - read operations are preceded by **r_lock** (aka **SHARED LOCK**) and followed by **unlock**
 - write operations are preceded by **w_lock** (aka **EXCLUSIVE LOCK**) and followed by **unlock**
 - Note: unlocking can be **delayed** w.r.t. to the end of the read/write operation
- Transactions that first read and then write an object may:
 - Acquire a **w_lock** already when reading
 - Acquire a **r_lock** first and then upgrade it into a **w_lock** (lock escalation)
- Possible states of an object:
 - **free**
 - **r-locked** (locked by one or more readers)
 - **w-locked** (locked by a writer)

Behavior of the Lock Manager (Conflict Table)

- The lock manager receives requests from the transactions and grants resources according to the conflict table
 - (in the table, we assume the request is made by a different transaction from at least one of those holding the lock)
 - If the lock request is granted, the resource is acquired
 - When an unlock is executed, the resource becomes available

Request	Resource status		
	FREE	R_LOCKED	W_LOCKED
r_lock	OK R_LOCKED	OK R_LOCKED (n++)	NO W_LOCKED
w_lock	OK W_LOCKED	NO R_LOCKED	NO W_LOCKED
unlock	ERROR	OK DEPENDS (n--)	OK FREE

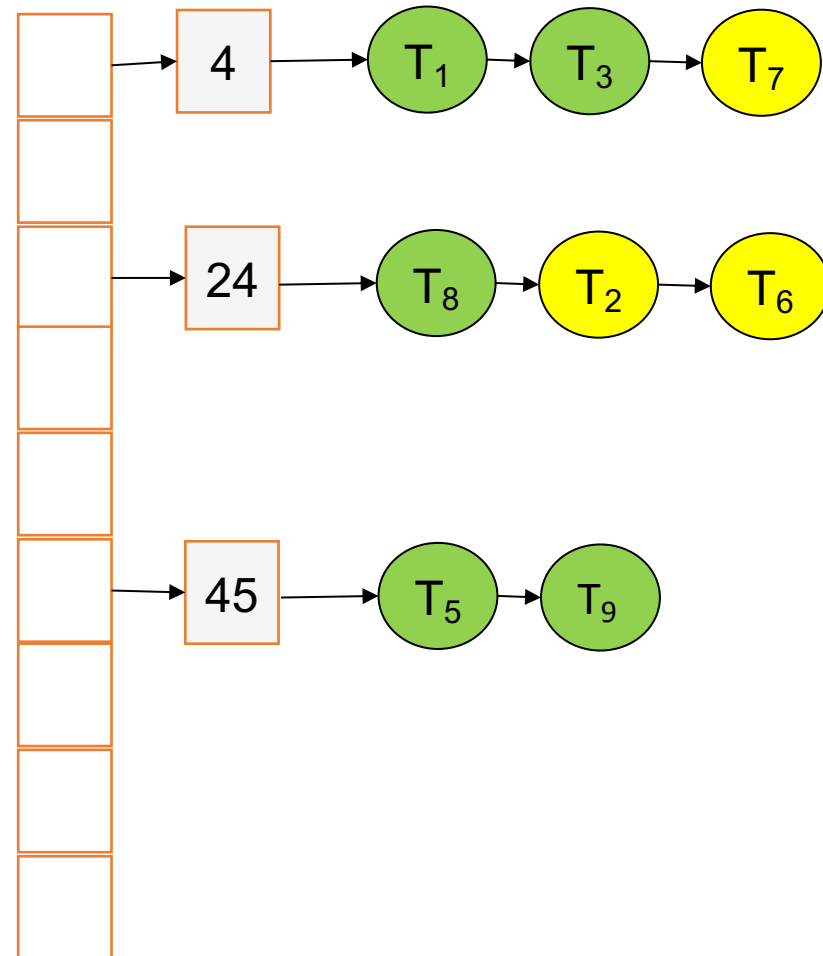
n: counter of the concurrent readers, (inc|dec)rement at each (r_|un)lock

Example

- **Arrival sequence:** $r_1(x)$, $w_1(x)$, $r_2(x)$, $r_3(y)$, $w_1(y)$
- $r_1(x)$
 - r_1 -lock(x) request $\rightarrow$ OK $\rightarrow$ x state = r-locked, $n_x=1$
- $w_1(x)$
 - w_1 -lock(x) request $\rightarrow$ OK (upgrade) $\rightarrow$ x state = w-locked
- $r_2(x)$
 - r_2 -lock(x) request $\rightarrow$ NO because w-locked $\rightarrow$ T_2 waits
- $r_3(y)$
 - r_3 -lock(y) request $\rightarrow$ OK $\rightarrow$ y state = r-locked, $n_y=1$ T_3 unlock(y) $\rightarrow$ $n_y=0$ y state = free
- $w_1(y)$
 - w_1 -lock(y) request $\rightarrow$ OK $\rightarrow$ y state = w-locked, T_1 unlock(x) $\rightarrow$ $n_x=0$ x status = free, T_1 unlock(y) $\rightarrow$ y status = free
 - $r_2(x)$ was waiting for r_2 -lock(x) request $\rightarrow$ OK
 x state = r-locked, $n_x=1$...
- **A posteriori schedule:** $r_1(x)$, $w_1(x)$, $r_3(y)$, $w_1(y)$, $r_2(x)$
- Note: the schedule depends on when T_1 actually unlocks x . Several a posteriori schedules are possible
- T_2 is **delayed**

How are locks implemented

- Typically by lock tables, which are hash tables indexing the lockable items via hashing
- Each locked item has a linked list associated with it
- Every node in the linked list represents the transaction that requested the lock, lock mode (SL/XL) and current status (granted/waiting).
- Every new lock request for the data item is appended as a new node to the list.
- Locks can be applied on both data and index records

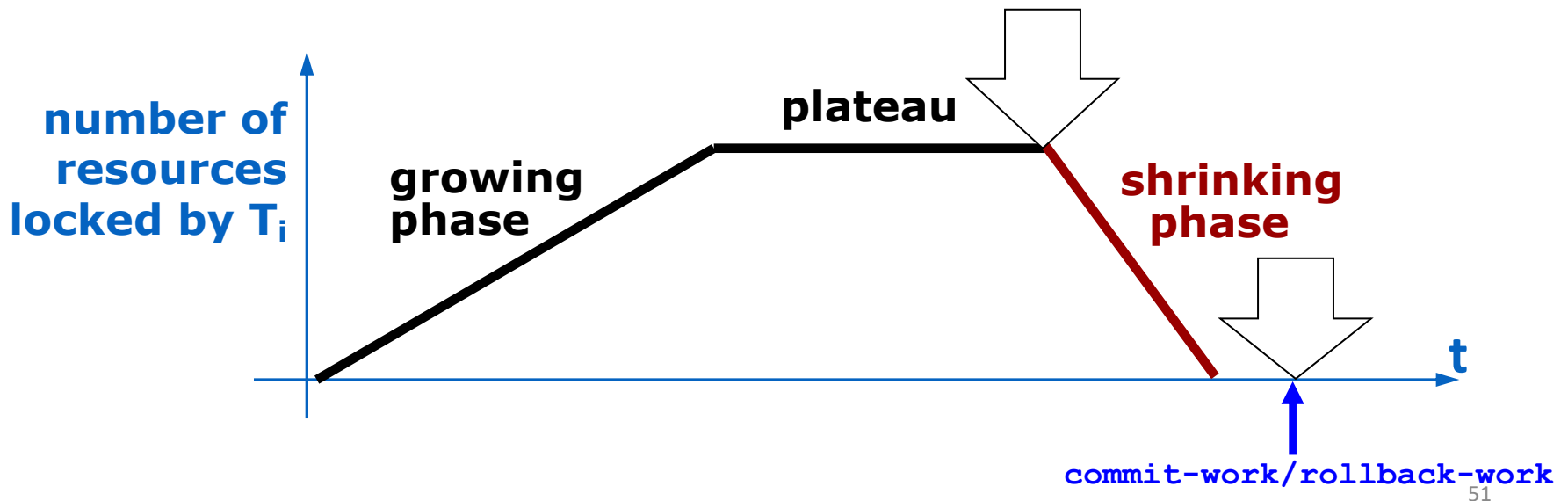


Is respecting locks enough for serializability?

- Arrival sequence $r_1(x), r_2(x), w_2(x), r_1(x)$
 - $r_1(x)$
 - r_1 -lock(x) request $\rightarrow$ OK $\rightarrow$ x state = r-locked, $n=1$ **T_1 unlock(x)** $\rightarrow$ x state = free, $n=0$
 - $r_2(x)$
 - r_2 -lock(x) request $\rightarrow$ OK $\rightarrow$ x state = r-locked, $n=1$
 - $w_2(x)$
 - w_2 -lock(x) request $\rightarrow$ OK (upgrade) $\rightarrow$ x state = w-locked
 - T_2 unlock(x) $\rightarrow$ x state = free, $n=0$
 - $r_1(x)$
 - **r_1 -lock(x)** request $\rightarrow$ OK $\rightarrow$ x state = r-locked, $n=1$..
- But the schedule mapped by the lock manager does not eliminate T_1 's **non-repeatable read**
- T_1 behavior: 1) releases its read lock on x too quickly; 2) acquires another lock after releasing one lock

Two-Phase Locking (2PL)

- Requirement (two-phase rule):
 - A transaction cannot acquire any other lock after releasing a lock
 - Sufficient to prevent non-repeatable reads (and more)
 - serializability!



Serializability

- Consider a scheduler that
 - Only processes well-formed transactions
 - Grants locks according to the conflict table
 - Checks that all transactions apply the two-phase rule
- The class of generated schedules is called **2PL**
- Result: schedules in 2PL are both view- and conflict-serializable
- **$(VSR \supset CSR \supset 2PL)$**

Assessing 2PL membership

- The scheduler normally receives an arrival sequence
 - With also lock and unlock requests
- The arrival sequence is transformed into a (possibly different) schedule (a-posteriori sequence)
- When checking 2PL membership, we are focusing on the a-posteriori sequence
 - Without the lock and unlock request
- So when we wonder whether a “schedule” is 2PL, we ask:
 - Does it correspond to an arrival sequence to which we can add lock and unlock requests compatible with 2PL that would leave the sequence unchanged (without making any transaction wait)?
- Under this interpretation, we again do not distinguish between the arrival sequence and the a-posteriori sequence

2PL implies CSR

- $\text{CSR} \supset \text{2PL}$: Every 2PL schedule is also conflict-serializable
- $\text{2PL} \rightarrow \text{CSR}$:
 - Suppose a 2PL schedule S is not CSR
 - S 's graph must contain at least one cycle $T_i \rightarrow T_j \rightarrow T_i$
 - Therefore there must be a pair of conflicting operations in reverse order, suppose that the conflictual operations appear as follows in the schedule
 - $OP^h_i(x), OP^k_j(x) \dots OP^u_j(y), OP^v_i(y)$ where (at least) one of $OP^h_i(x), OP^k_j(x)$ is a write and (at least) one of $OP^u_j(y), OP^v_i(y)$ is a write
 - $OP^h_i(x), OP^k_j(x)$
 - T_i must have released a lock for T_j to access x
 - Later in the schedule... $OP^u_j(y), OP^v_i(y)$
 - T_i must have acquired a lock for the conflict to occur $\rightarrow$ CONTRADICTION
 - Inclusion of 2PL in VSR descends from $\text{VSR} \supset \text{CSR}$
 - This proves that all 2PL schedules are view-serializable too

2PL implies CSR (alternative proof)

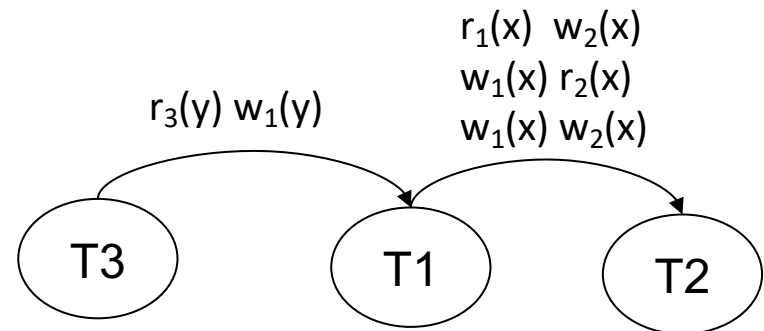
- 2PL $\rightarrow$ CSR: We assume that a schedule S is 2PL
- Consider, for each transaction, the end of the plateau
 - (i.e., the moment in which it holds all locks and is going to release the first one)
- We sort (and re-label) the transactions by this temporal value and consider the corresponding serial schedule S'
- In order to prove (by contradiction) that S is conflict-equivalent to S' , $S' \approx_c S$, ... (in all conflicting pairs, transactions occur in the same order)

2PL implies CSR (alternative proof)

- Consider a (generic) conflict $o_i \rightarrow o_j$ in S' with $o_i \in T_i$ $o_j \in T_j$ $i < j$
- By definition of conflict, o_i and o_j address the same resource r , and at least one of them is a write
- Can o_i and o_j occur in reverse order in S ?
 - No, because then T_j should have released r before T_i could acquire it
 - This contradicts the ordering criterion (T_j should have started releasing locks before T_i)
- Inclusion of 2PL in VSR descends from $VSR \supset CSR$
- This proves that all 2PL schedules are view-serializable too
- And they can be checked with negligible overhead

2PL smaller than CSR

- $\text{CSR} \supset \text{2PL}$: Every 2PL schedule is also conflict-serializable, but the converse is not necessarily true
- Counter-example: $r_1(x) \ w_1(x) \ r_2(x) \ w_2(x) \ r_3(y) \ w_1(y)$
 - It violates 2PL
 - $r_1(x) \ w_1(x) \searrow r_2(x) \ w_2(x) \ r_3(y) \nearrow w_1(y)$
 - T_1 releases T_1 acquires
 - However, it is conflict-serializable: $T_3 < T_1 < T_2$
 - On x: $r_1 \ w_1 \ r_2 \ w_2$
 - On y: $r_3 \ w_1$



A visualization of the 2PL test

Resources on the vertical axis and operation times on the horizontal axis

	1	2	3	4	5	6
x	$\nearrow_1 r_1$	$\nearrow_1 w_1$	$\downarrow_1$	$\nearrow_2 r_2$	$\nearrow_2 w_2$	$\downarrow_2$
y				$\nearrow_3 r_3$	$\downarrow_3$	$\nearrow_1 w_1$
				2	3	

$\nearrow$: read lock request

$\nearrow$: write lock request

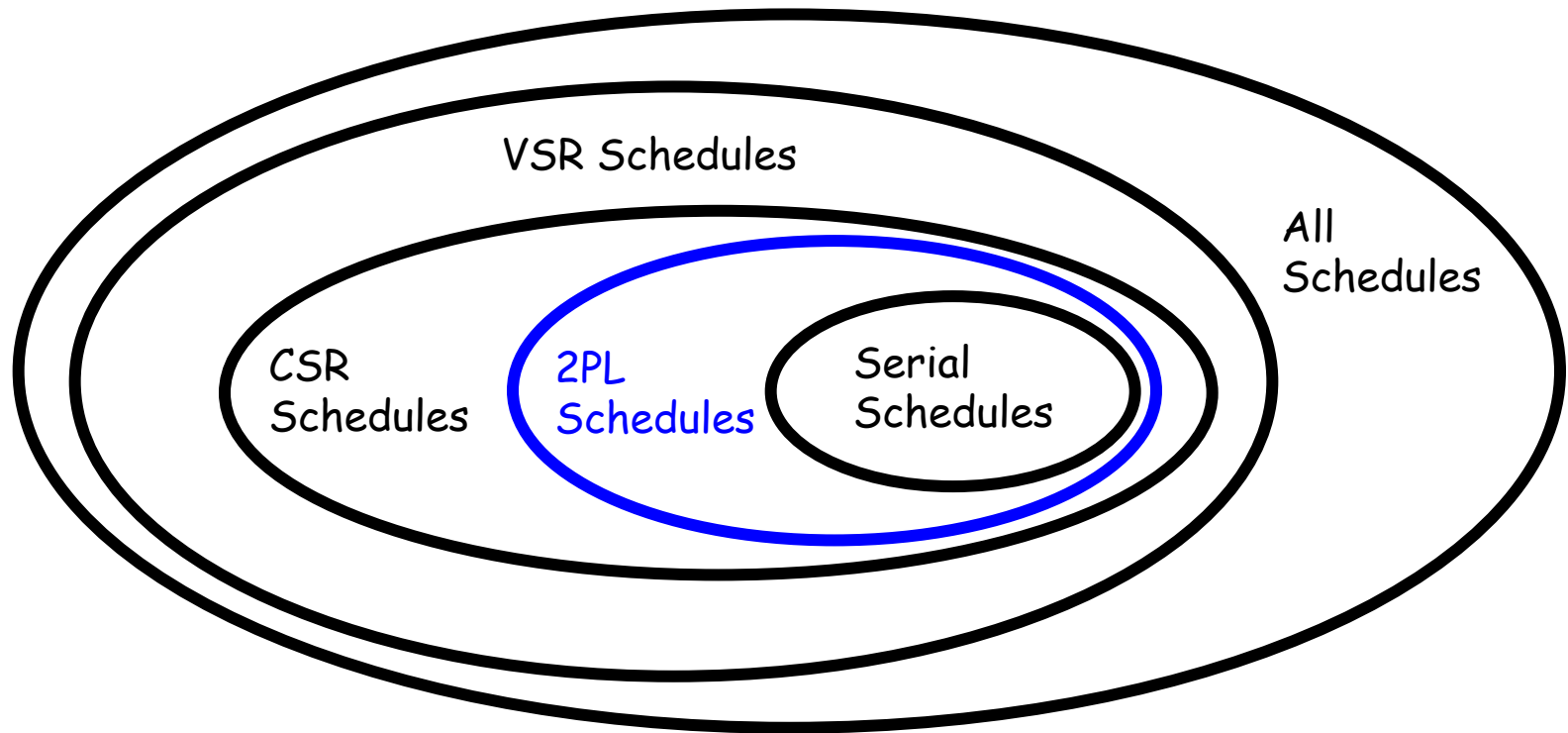
$\nearrow$: lock upgrade

$\downarrow$: unlock request

2PL and other anomalies

- Non-repeatable read: $r_1 - r_2 - w_2 - r_1$
 - Already shown
- Lost update: $r_1 - r_2 - w_2 - w_1$
 - T_1 releases a lock to T_2 and then tries to acquire another one
- Phantom update: $r_1 - r_2 - w_2 - r_1$
 - T_1 releases a lock to T_2 and then tries to acquire another one
- Phantom insert: $r_1 - w_2(\text{new data}) - r_1$
 - T_1 releases a lock to T_2 and then tries to acquire another one
(NOTE: T_2 does not necessarily write on data already locked by T_1 → requires lock on “future data” aka predicate locks)
- Dirty read: $r_1 - w_1 - r_2 - \text{abort}_1 - w_2$
 - Requires dealing with abort

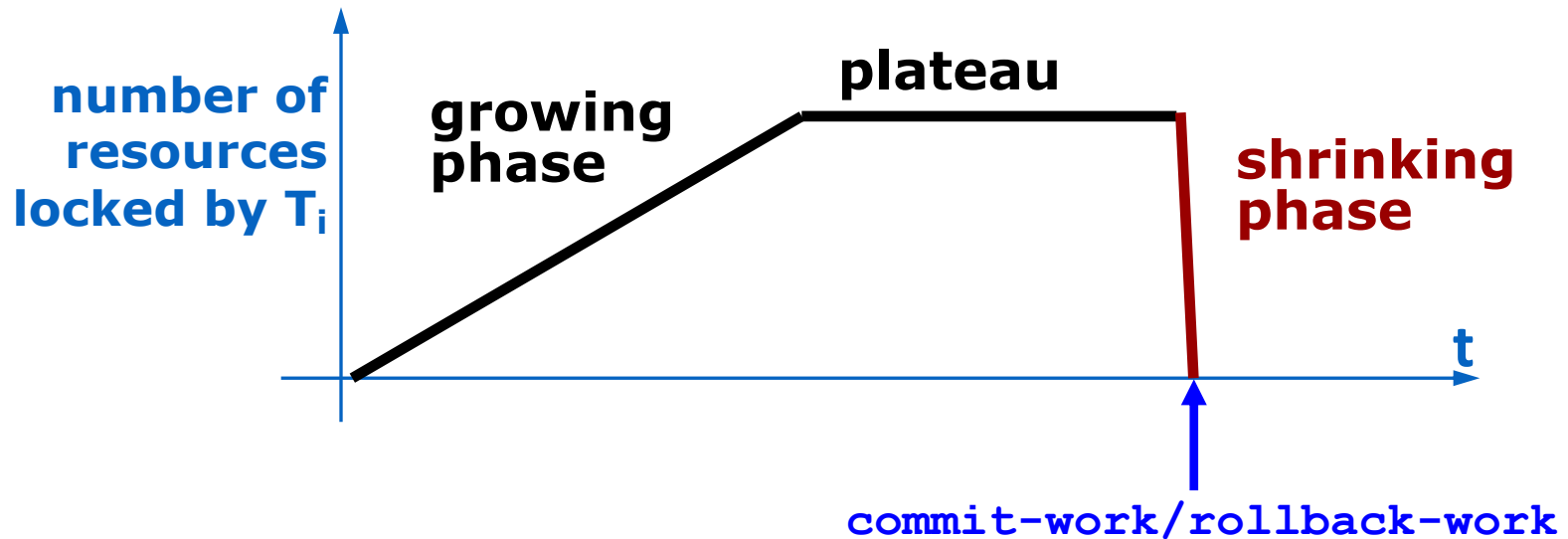
CSR, VSR and 2PL



Dirty reads are still a menace: Strict 2PL

- Up to now, we were still using the hypothesis of commit-projection (no transactions in the schedule abort)
 - 2PL, as seen so far, does not protect against dirty (uncommitted data) reads (and therefore neither do VSR nor CSR)
 - Releasing locks before rollbacks exposes “dirty” data
- To remove this hypothesis, we need to add a constraint to 2PL, which defines **strict 2PL**:
 - Locks held by a transaction can be released only after commit/rollback
 - Remember: rollback restores the state prior to the aborted updates
- This version of 2PL is used in most commercial DBMSs whenever a high level of isolation is required (see next: SQL isolation levels)

Strict 2PL in Practice



- Strict 2PL locks are also called **long duration** locks, as opposed to **short duration** locks of 2PL
- Note: real systems may apply 2PL policies differently to read and write locks
- Typically: long duration strict 2PL write locks, variable policies for read locks
- NOTE: long duration read locks are costly in terms of performances: real systems replace them with more complex mechanisms

How to prevent phantom inserts: predicate locks

- A phantom insertion occurs when a transaction adds items to a dataset previously read by another transaction
- To prevent phantom inserts a lock should be placed also on “future data”, i.e., inserted data that **would satisfy** a previous query
- **Predicate locks** extend the notion of **data locks** to “future data”
- A transaction T = update Tab set B=1 where $A < 1$
- Then, the lock is on predicate $A < 1$
 - Other transactions cannot insert, delete, or update any tuple satisfying this predicate
 - In the worst case (predicate locks not supported):
 - The lock extends to the entire table
 - In case the implementation supports predicate locks:
 - The lock is managed with the help of indexes (gap lock)

Tab	
A	B

Isolation Levels in SQL:1999 (and JDBC)

- SQL defines transaction isolation levels which specify the anomalies that should be prevented by running at that level
- The level does not affect write locks. A transaction should always get an exclusive lock on any data it modifies, and hold it until completion (strict 2PL on write locks), regardless of the isolation level
- For read operations, levels define the degree of protection from the effects of modifications made by other transactions

Why long duration write locks are necessary

- Consider the following schedule (admissible if write locks are short duration) and **remove the hypothesis that aborts do not occur**
- $w_1(x) \dots w_2(x) \dots ((c_1 \text{ or } a_1) \text{ and } (c_2 \text{ or } a_2))$ in any order
 - T_2 is allowed to write over the same object updated by T_1 which has not yet completed
- If T_1 aborts... e.g., $w_1(x) \dots w_2(x) \dots, a_1, (c_2 \text{ or } a_2)$
 - How to process event a_1 ?
 - If x is restored to the state before T_1 , T_2 's update is lost, so if T_2 commits x has a stale value
 - If x is NOT restored and T_2 also aborts... then T_2 's proper before state cannot be reinstalled either!
- Thus: write locks are held until the completion of the transaction to enable the proper processing of abort events
- The anomaly of the above non commit-projection schedule is named **dirty write**

Isolation Levels in SQL:1999 (and JDBC)

- READ UNCOMMITTED allows dirty reads, non-repeatable reads and phantom updates and inserts:
 - No read locks (and ignores locks of other transactions)
- READ COMMITTED prevents dirty reads but allows non-repeatable reads and phantom updates/inserts:
 - Read locks (and complies with locks of other transactions), but without 2PL on read locks (read locks are released as soon as the read operation is performed and can be acquired again)
- REPEATABLE READ avoids dirty reads, non-repeatable reads and phantom updates, but allows phantom inserts:
 - long duration read locks → Strict 2PL also for reads
- SERIALIZABLE avoids all anomalies:
 - Strict 2PL with predicate locks to avoid phantom inserts
- Note that SQL standard isolation levels dictate minimum requirements, real systems may go beyond (e.g., in MySQL and Postgres REPEATABLE READ avoids phantom inserts too) and use different mechanisms (e.g., to avoid long duration read locks)
- What about lost updates? Different systems may offer different guarantees

	Dirty Read	Non-rep. read	Phantoms
Read uncommitted	Y	Y	Y
Read committed	N	Y	Y
Repeatable read	N	N	Y (insert)
Serializable	N	N	N

SQL92 serializable <> serial !

- Serializable transactions don't necessarily execute serially
- The requirement is that transactions can only commit if the result would be as if they had executed serially in any order
- The locking requirements to meet this guarantee can frequently lead to a **deadlock** where one of the transactions needs to be rolled back
- Therefore, the SERIALIZABLE isolation level is used **sparingly** and is NOT the default in most commercial systems

SQL isolation levels and locks

- SQL Isolation levels may be implemented with the appropriate use of locks
- Commercial systems make joint use of locks and of timestamp-based concurrency control mechanisms

	READ LOCKS	WRITE LOCKS
READ UNCOMMITTED	Not required	Well formed writes Long duration write locks
READ COMMITTED	Well formed reads Short duration read locks (data and predicate)	Well formed writes Long duration write locks
REPEATABLE READ	Well formed reads Long duration data read locks Short duration predicate read locks	Well formed writes Long duration write locks
SERIALIZABLE	Well formed reads Long duration read locks (predicate and data)	Well formed writes Long duration write locks

Setting transaction characteristics in SQL

`<set transaction statement> ::=`

`SET [ LOCAL ] TRANSACTION <transaction characteristics>`

`<transaction characteristics> ::= [<transaction mode> [{ <comma>
 <transaction mode> }...]]`

`<transaction mode> ::= <isolation level> | <transaction access
mode>`

`| <diagnostics size>`

`<transaction access mode> ::= READ ONLY | READ WRITE`

`<isolation level> ::= ISOLATION LEVEL <level of isolation>`

`<level of isolation> ::= READ UNCOMMITTED | READ COMMITTED
 | REPEATABLE READ | SERIALIZABLE`

The impact of locking: waiting is dangerous!

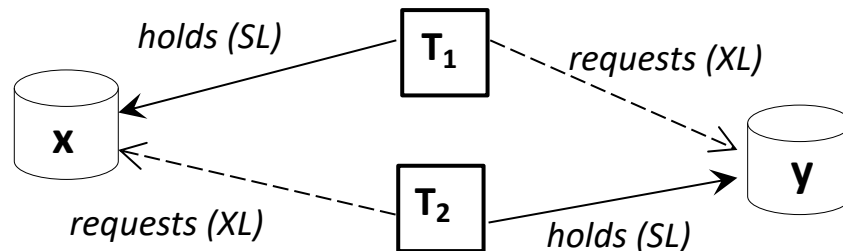
- Locks are tracked by lock tables (main memory data structures)
 - Resources can be Free, or Read-Locked, or Write-locked
 - To keep track of readers, every resource also has a “read counter”
- Transactions requesting locks are either **granted the lock** or **suspended and queued** (first-in first-out). There is risk of:
 - **Deadlock**: two or more transactions in endless (mutual) wait
 - Typically occurs when each transaction waits for another to release a lock (in particular: $r_1 r_2 w_1 w_2 \rightarrow$ see update lock later)
 - **Starvation**: a single transaction in endless wait
 - Typically occurs due to write transactions waiting for resources that are continuously read (e.g., index roots)

Deadlock

- Occurs because concurrent transactions hold and in turn request resources held by other transactions

- $T_1 : r_1(x) \ w_1(y)$

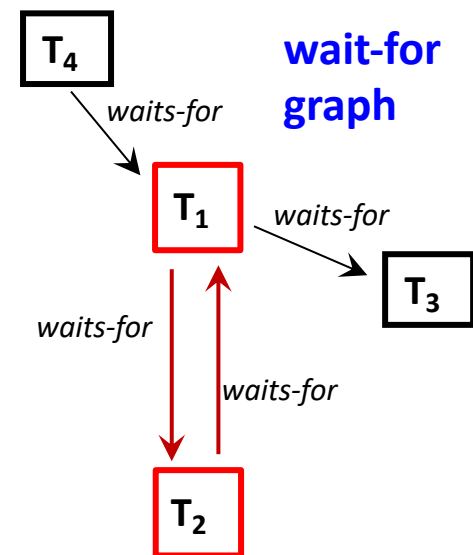
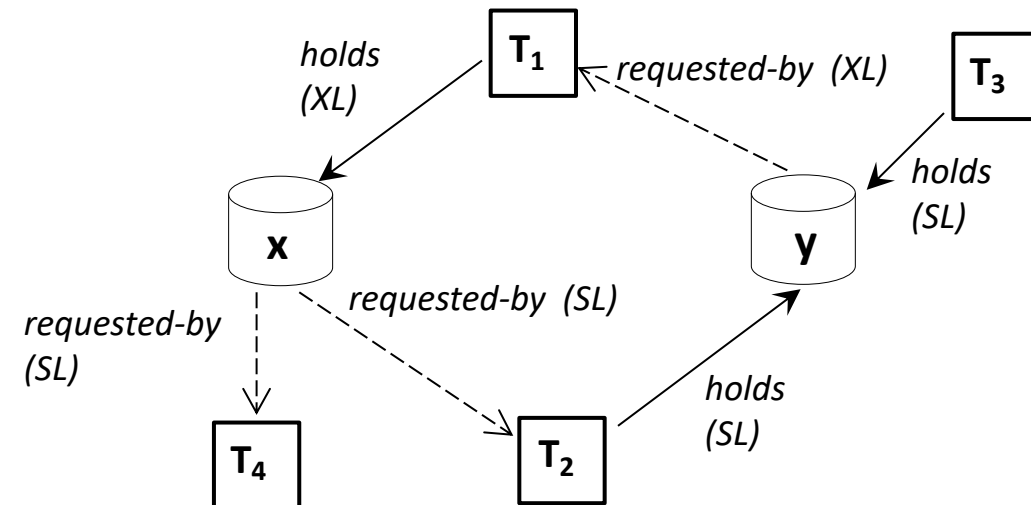
- $T_2 : r_2(y) \ w_2(x)$



- $S : r_lock_1(x), r_lock_2(y), r_1(x), r_2(y), w_lock_1(y), w_lock_2(x) \rightarrow \text{deadlock}$

Deadlock

- **Lock graph**: a bipartite graph in which nodes are resources or transactions and arcs are lock requests or lock assignments
- **Wait-for graph**: a graph in which nodes are transactions and arcs are “waits for” relationships
- A deadlock is represented by a cycle in the wait-for graph of transactions



Deadlock Resolution Techniques

- **Timeout**

- Transactions killed after a long wait
 - How long?

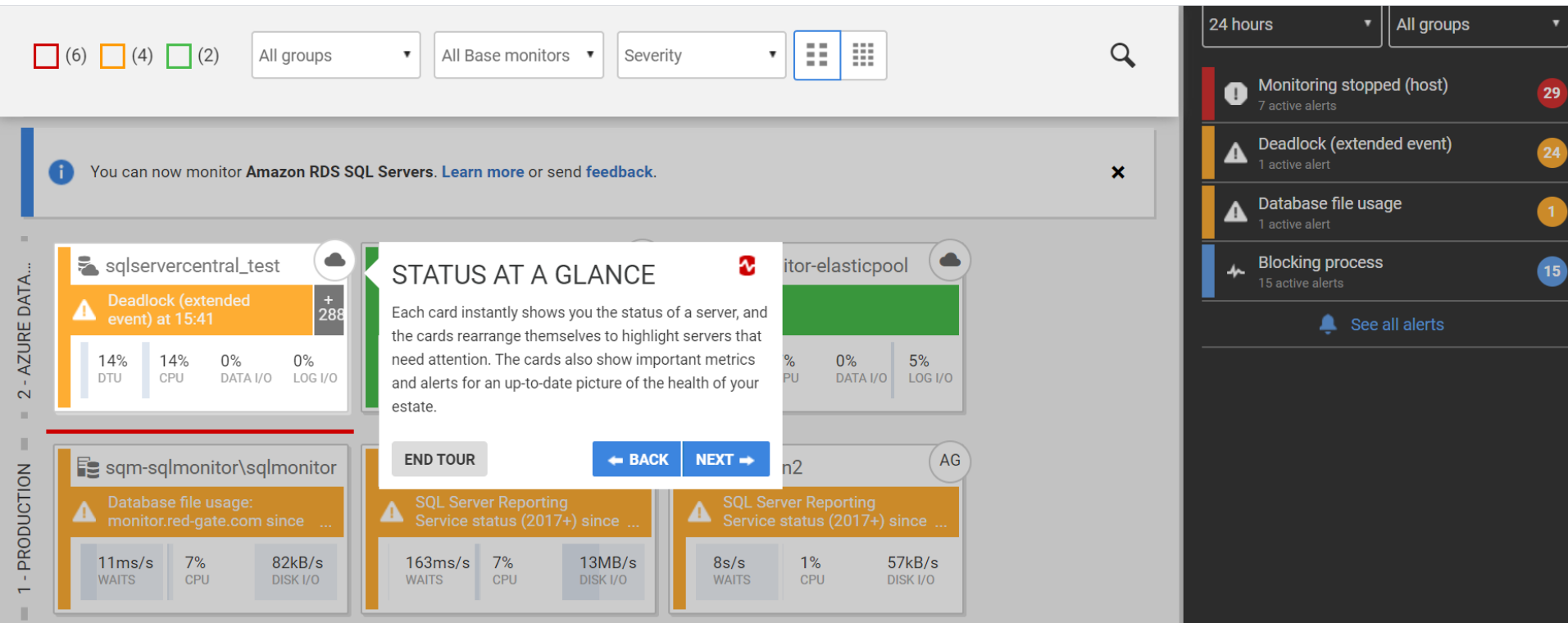
- **Deadlock prevention**

- Transactions killed when they COULD BE in a deadlock
 - Heuristics

- **Deadlock detection**

- Transactions killed when they ARE in a deadlock
 - Inspection of the wait-for graph

Deadlock detection in commercial dashboards



Timeout Method

- A transaction is killed and restarted after a given amount of waiting (assumed as due to a deadlock)
 - The simplest method, widely used in the past
- The timeout value is system-determined (sometimes it can be altered by the database administrator)
- The problem is choosing a proper timeout value
 - Too long: useless waits whenever deadlocks occur
 - Too short: unrequired kills, redo overhead
- <http://davebland.com/how-often-does-sql-server-look-for-deadlocks>

Deadlock Prevention

- Idea: killing transactions that could cause cycles
- **Resource-based prevention:** Restrictions on lock requests
 - Transactions request all resources at once, and only once
 - Resources are globally sorted and must be requested “in global order”
 - Problem: it’s not easy for transactions to anticipate all requests!
- **Transaction-based prevention:** restrictions based on transactions’ IDs
 - Assigning IDs to transactions incrementally → transactions’ “age”
 - Preventing “older” transactions from waiting for “younger” ones to end their work
 - Options for choosing the transaction to kill
 - **Preemptive** (killing the holding transaction – wound-wait)
 - **Non-preemptive** (killing the requesting transaction – wait-die)
 - Problem: too many “killings”! (waiting probability $\gg$ deadlock probability)

Deadlock Detection

- Requires an algorithm to detect cycles in the wait-for graph
 - Must work with **distributed** resources efficiently & reliably

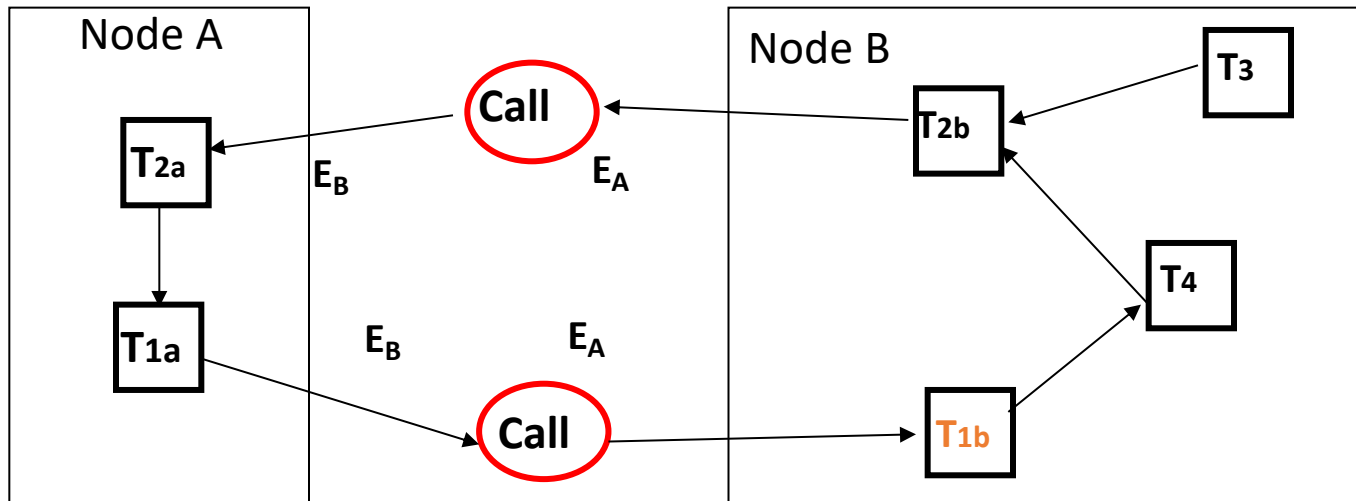
- An elegant solution: **Obermarck's** algorithm (DB2-IBM):

Ron Obermarck: *Distributed Deadlock Detection Algorithm*.
ACM Trans. Database Syst. 7(2): 187-208 (1982)

- Assumptions
 - Transactions execute on a single main node (one locus of control)
 - Transactions may be decomposed in “sub-transactions” running on other nodes
 - When a transaction spawns a sub-transaction it suspends work until the latter completes
 - Two wait-for relationships:
 - T_i waits for T_j on the same node because T_i needs a datum locked by T_j
 - A sub-transaction of T_i waits for another sub-transaction of T_i running on a different node (via external call E)

Distributed Deadlock Detection

Distributed dependency graph: external call nodes represent a sub-transaction activating another sub-transaction at a different node

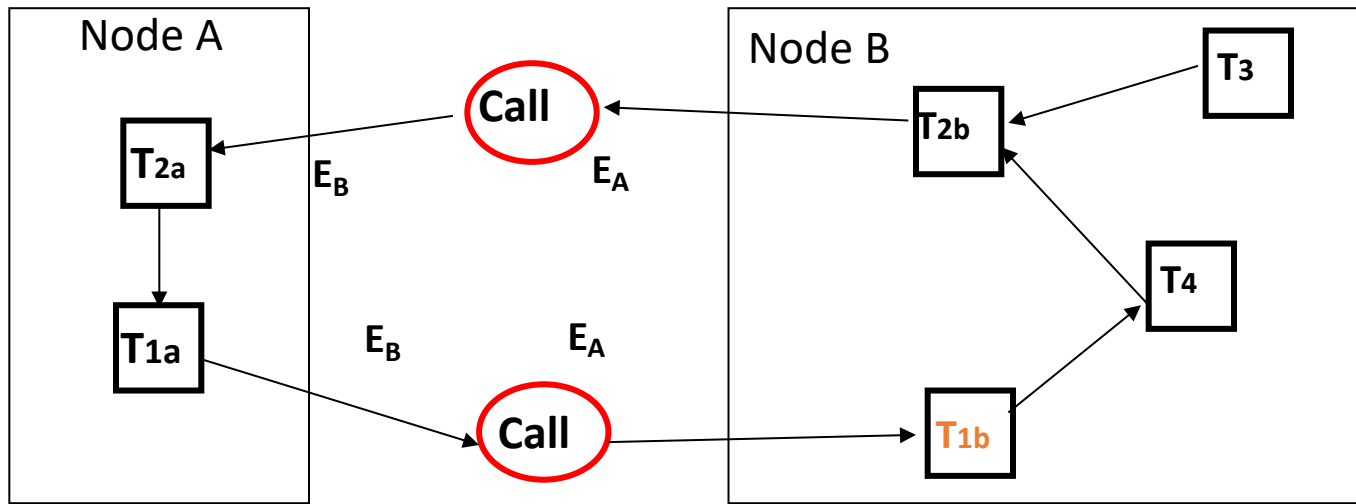


- Representation of the status
 - at Node A: $E_b \rightarrow T_{2a} \rightarrow T_{1a} \rightarrow E_b$
 - at Node B: $E_a \rightarrow T_{1b} \rightarrow T_{2b} \rightarrow E_a$
- NOTATION: symbol $\rightarrow$: 1) “wait-for” relation among local transactions 2) if one term is an external call, either the source is waited for by a remote transaction or the sink waits for a remote transaction
- Potential Deadlock: T_{2a} waits for T_{1a} (data lock) that waits for T_{1b} (call) that waits for T_{2b} (data locks) that waits for T_{2a} (call): cycle!
- Problem: how to detect such an occurrence without maintaining the global view

Obermarck's Algorithm

- Goal: detection of a potential deadlock looking only at the local view of a node
- Method: establishing a communication protocol whereby each node has a local projection of the global dependencies
- Nodes exchange information and update their local graph based on the received information
- Communication is optimized to avoid that multiple nodes detect the same potential deadlock
- Node A sends its local info to a node B only if
 - A contains a transaction T_i that is waited for from another remote transaction and waits for a transaction T_j , in turn waiting for another transaction active on B
 - $i > j$ (this ensure a kind of message “forwarding” along a node path where node A “precedes” node B if $i > j$)
 - Mnemonically: I send info to you if a distributed transaction listed at me waits for a distributed transaction listed at you with **smaller** index

Forwarding rule

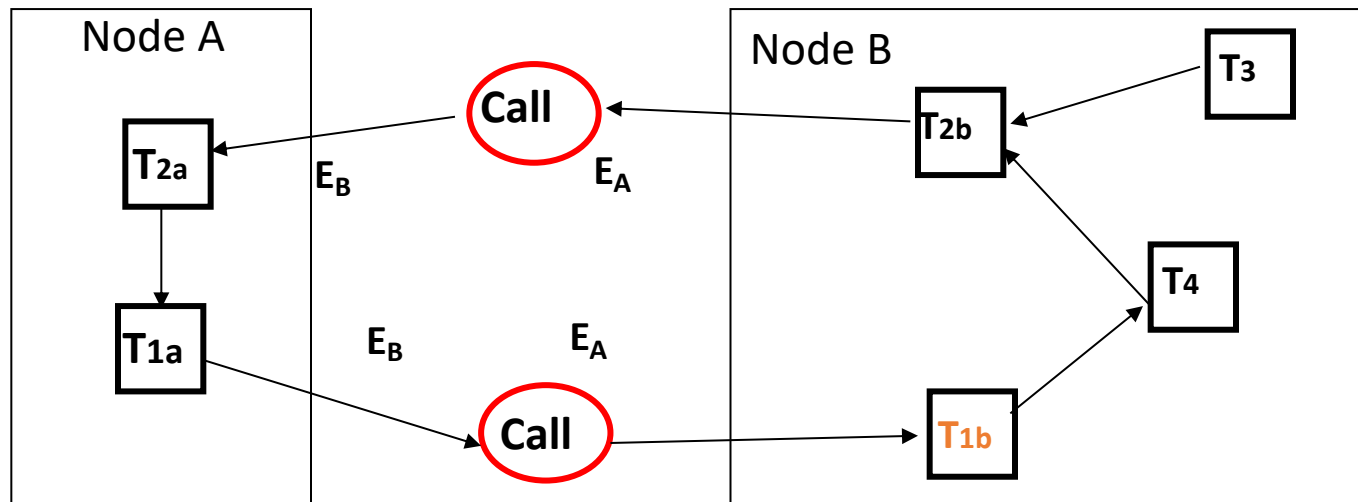


- Node A:
 - Activation/wait sequence: $E_b \rightarrow T_2 \rightarrow T_1 \rightarrow E_b$
 - $i=2, j=1$
 - A can dispatch info to B
- Node B:
 - Activation/wait sequence (only distributed transactions count): $E_a \rightarrow T_1 \rightarrow T_2 \rightarrow E_a$
 - $i=1, j=2$
 - B does not dispatch info to A

Obermarck's Algorithm

- Runs periodically at each node
- Consists of 4 steps
 - Get graph info (wait dependencies among transactions and external calls) from the “previous” nodes. Sequences contain only node and top-level transaction identifiers
 - Update the local graph by merging the received information
 - Check the existence of cycles among transactions denoting potential deadlocks: if found, select one transaction in the cycle and kill it
 - Send updated graph info to the “next” nodes

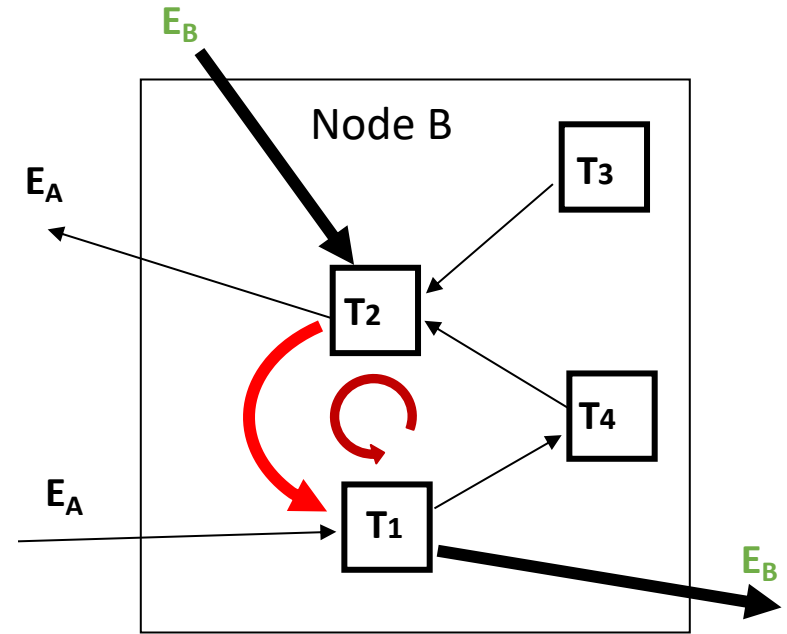
Algorithm execution, step 1: communication



- Distributed Deadlock Detection: forwarding rule
 - at Node A: $E_b \rightarrow T_2 \rightarrow T_1 \rightarrow E_b$ info sent to Node B
 - at Node B: $E_a \rightarrow T_1 \rightarrow T_2 \rightarrow E_a$ info not sent ($i < j$)

Algorithm execution, step 2: local graph update

- at Node B:
- 1. $E_b \rightarrow T_2 \rightarrow T_1 \rightarrow E_b$ is received
- 2. $E_b \rightarrow T_2 \rightarrow T_1 \rightarrow E_b$ is added to the local wait-for graph
- 3. Deadlock detected (cycle among T_1 and T_2):
 - T_1 or T_2 or T_4 killed (rollback)

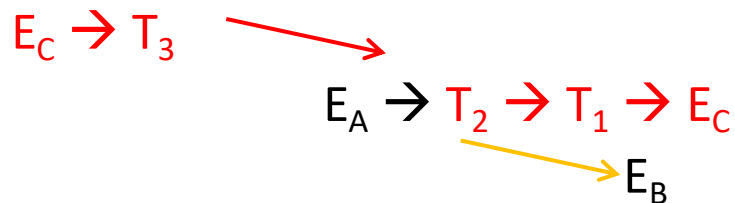


Another example

- Initially:
 - at Node A, $E_C \rightarrow T_3 \rightarrow T_2 \rightarrow E_B$
 - at Node B, $E_A \rightarrow T_2 \rightarrow T_1 \rightarrow E_C$
 - at Node C, $E_B \rightarrow T_1 \rightarrow T_3 \rightarrow E_A$
- Node A: $3 > 2$ can send (to B)
- Node B: $2 > 1$ can send (to C)
- Node C: $1 < 3$ cannot send
- We assume that node A is the first to execute, sending its sequence to B (i.e., to the node hosting the (sub)transaction that keeps A's transaction on hold)

Another example, continued

- At node B:
- $E_A \rightarrow T_2 \rightarrow T_1 \rightarrow E_C$ is merged with $E_C \rightarrow T_3 \rightarrow T_2 \rightarrow E_B$
- Result:



- A new potential deadlock $E_C \rightarrow T_3 \rightarrow T_1 \rightarrow E_C$ is found by combining the $E_C \rightarrow T_3 \rightarrow T_2 \rightarrow E_B$ message with the local info $E_A \rightarrow T_2 \rightarrow T_1 \rightarrow E_C$. This can be sent to node C ($3 > 1$)
- At node C:
- Local info $E_B \rightarrow T_1 \rightarrow T_3 \rightarrow E_A$ is merged with message $E_C \rightarrow T_3 \rightarrow T_1 \rightarrow E_C$

$$E_B \rightarrow T_1 \leftarrow \rightarrow T_3 \rightarrow E_A$$
- The deadlock is found and either T_1 or T_3 is killed.

Oberman: immateriality of conventions

- There are two arbitrary choices in the algorithm:
- Send messages only if: (1) $i > j$ vs. (2) $i < j$
- Send them to: (a) the following node vs. (b) the preceding node
- Therefore, there are four versions/variants of the algorithm
 - (1+a), (1+b), (2+a), (2+b)
 - The sequence of the sent messages is different
 - However, they all identify deadlocks (if present)

Deadlocks in practice

- Their probability is much less than the conflict probability
 - Consider a file with n records and two transactions doing two accesses to their records (uniform distribution); then:
 - Conflict probability is $O(1/n)$: $\sum_{i=1..n} (1/n * 1/n) = n * (1/n * 1/n) = 1/n$
 - Deadlock probability is $O(1/n^2)$: T_1 conflicts with T_2 AND vice versa
- Still, they do occur (once every minute in a mid-size bank)
 - The probability is linear in the number of transactions, **quadratic in their length** (measured by the number of lock requests)
 - Shorter transactions are healthier (ceteris paribus)
- There are techniques to limit the frequency of deadlocks
 - Update Lock, Hierarchical Lock, ...,

Update lock

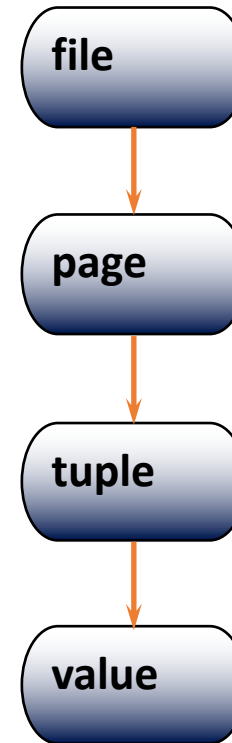
- The most frequent deadlock occurs when 2 concurrent transactions start by reading the same resources (SL) and then decide to write and try to upgrade their lock to XL
- To avoid this situation, systems offer the UPDATE LOCK (UL) – asked by transactions that will **read and then write**

Request	Resource status			
	free	SL	UL	XL
SL	OK	OK	OK	No
UL	OK	OK	No	No
XL	OK	No	No	No

- Update locks are easy to implement and mitigate the most frequent cause of collision: $r_1(x) \ r_2(x) \ w_1(x) \ w_2(x)$
- They are requested by using SQL SELECT **FOR UPDATE** statement

Hierarchical Locking

- Update locks prudentially extend the interval during which a resource is locked
- What to lock? An entire table? → reduces concurrency too much
- Locks can be specified with different granularities
 - e.g.: schema, table, fragment, page, tuple, field
- Objectives:
 - Locking the minimum amount of data
 - Recognizing conflicts as soon as possible
- Method: asking locks on hierarchical resources by
 - Requesting resources top-down until the right level is obtained
 - Releasing locks bottom-up



**Coarser
Granularity**

**Increased
Concurrency**

Intention Locking Scheme

- 5 Lock modes:
 - In addition to read (SHARED) locks (SL) and write (EXCLUSIVE) locks (XL)
- The new modes express the “**intention**” of locking at lower (finer) levels of granularity
 - **ISL**: Intention of locking a subelement of the current element in shared mode
 - **IXL**: Intention of locking a subelement of the current element in exclusive mode
 - **SIXL**: Lock of the element in shared mode with intention of locking a subelement in exclusive mode (SL+IXL)

Hierarchical Locking Protocol

- Locks are requested starting from the root (e.g., starting from the whole table) and going down in the hierarchy
- Locks are released starting from the leaves and going up in the hierarchy
- To request an SL or ISL lock on a non-root element, a transaction must hold an equally or more restrictive lock (ISL or IXL) on its “parent”
- To request an IXL, XL or SIXL lock on a non-root element, a transaction must hold an equally or more restrictive lock (SIXL or IXL) on its “parent”
- When a lock is requested on a resource, the lock manager decides based on the rules specified in the hierarchical lock granting table

Hierarchical lock granting table

Request	Resource state					
	free	ISL	IXL	SL	SIXL	XL
ISL	OK	OK	OK	OK	OK	No
IXL	OK	OK	OK	No	No	No
SL	OK	OK	No	OK	No	No
SIXL	OK	OK	No	No	No	No
XL	OK	No	No	No	No	No

Example

P ₁	P ₂
t ₁	t ₅
t ₂	t ₆
t ₃	t ₇
t ₄	t ₈

P ₁	P ₂
t ₁	t ₅
t ₂	t ₆
t ₃	t ₇
t ₄	t ₈

Root = TableX

Page 1 (P₁): t₁, t₂, t₃, t₄ tuples

Page 2 (P₂): t₅, t₆, t₇, t₈ tuples

Transaction 1: read(P₁)
 write(t₃)
 read(t₈)

Transaction 2: read(t₂)
 read(t₄)
 write(t₅)
 write(t₆)

They are NOT in r-w conflict!
 (independently of the order)

Lock Sequences

P ₁	P ₂
t ₁	t ₅
t ₂	t ₆
t ₃	t ₇
t ₄	t ₈

Transaction 1: **read(P₁)**
write(t₃)
read(t₈)

P ₁	P ₂
t ₁	t ₅
t ₂	t ₆
t ₃	t ₇
t ₄	t ₈

Transaction 2: **read(t₂)**
read(t₄)
write(t₅)
write(t₆)

T₁:

IXL(root)
 SIXL(P₁)
 XL(t₃)
 ISL(P₂)
 SL(t₈)

T₂:

IXL(root)
 ISL(P₁)
 SL(t₂)
 SL(t₄)
 IXL(P₂)
 XL(t₅)
 XL(t₆)

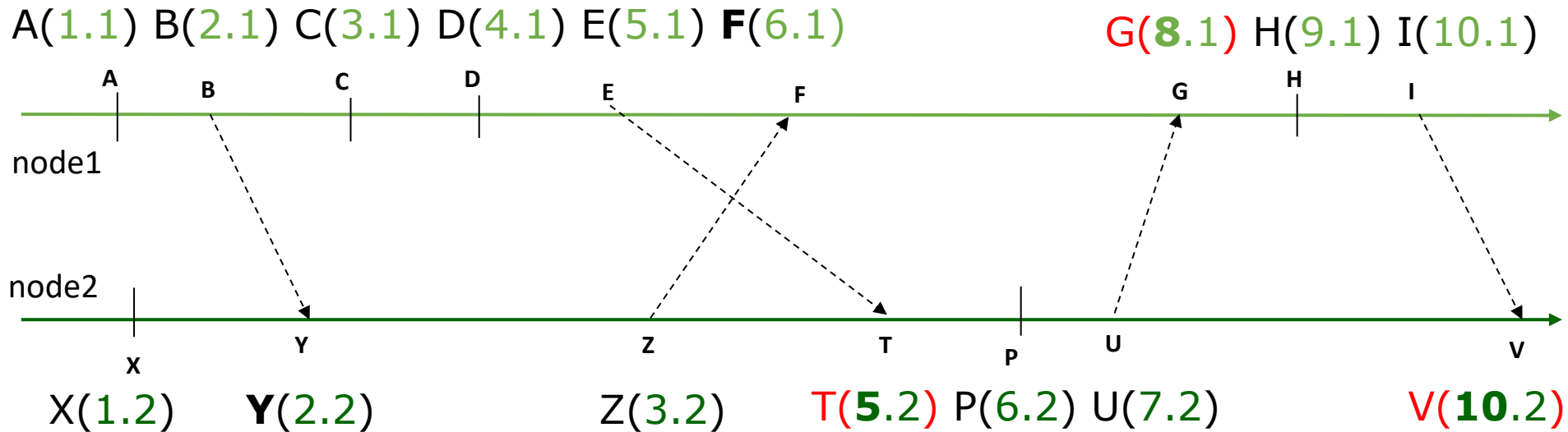
Concurrency Control Based on Timestamps

- Locking is also named **pessimistic** concurrency control because it assumes that collisions (transactions reading-writing the same object concurrently) will arise
- In reality collisions are rare
- Alternative and complementary to 2PL (and to locking in general) are **optimistic** concurrency control methods
- **Timestamp:**
 - Identifier that defines a total ordering of the events of a system
- Each transaction has a timestamp representing the time at which the transaction begins so that transactions can be ordered by “birth date”: smaller index → older transaction
- A schedule is accepted only if it reflects the serial ordering of the transactions induced by their timestamps

Assigning timestamps in distributed systems

- Timestamp: an indicator of the “current time”
- Assumption: no “global time” is available
- Mechanism: a system’s function gives out timestamps on requests
- Syntax: timestamp = event-id.node-id
 - event-ids are unique at each node
- Note that the notion of time is “lexical”: timestamp 5.1 “occurs before” timestamp 5.2
- Synchronization: send-receive of messages
 - for a given message m , $\text{send}(m)$ precedes $\text{receive}(m)$
- Algorithm (Lamport method): cannot receive a message from “the future”, if this happens the “bumping rule” is used to bump the timestamp of the receive event beyond the timestamp of the send event
- **Mnemonically**: if I receive a message from you that has a timestamp greater than my last emitted one I update my current timestamp to exceed yours

Example of timestamp assignment



- Events Y and F represent messages received “from the present or past” → OK the local timestamp is aligned without bumping
- Events T, G and V represent messages received “from the future” → The local timestamp is incremented (bumped) accordingly leaving a “hole” in the local event sequence
- Note that the timestamp of the receive events T, G and V is generated so as to exceed that of the send event: $G(8.1) > U(7.2)$

TS concurrency control principles

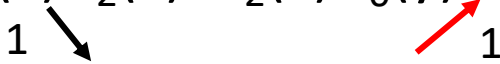
- The scheduler has two counters: $RTM(x)$ and $WTM(x)$ for each object
- The scheduler receives read/write requests tagged with the timestamp of the requesting transaction:
 - $r_{ts}(x)$:
 - If $ts < WTM(x)$ the request is **rejected** and the transaction is killed
 - Else, access is **granted** and we set $RTM(x) = \max(RTM(x), ts)$
 - $w_{ts}(x)$:
 - If $ts < RTM(x)$ or $ts < WTM(x)$ the request is **rejected** and the transaction is killed
 - Else, access is **granted** and we set $WTM(x) = ts$
- Many transactions are killed

Example

Assume $\text{RTM}(x) = 7$
 $\text{WTM}(x) = 4$

Request	Response	New value
$r_6(x)$	ok	
$r_8(x)$	ok	$\text{RTM}(x) = 8$
$r_9(x)$	ok	$\text{RTM}(x) = 9$
$w_8(x)$	no	T_8 killed
$w_{11}(x)$	ok	$\text{WTM}(x) = 11$
$r_{10}(x)$	no	T_{10} killed

2PL vs. TS

- They are incomparable
- Schedule in TS but not in 2PL
 - $r_1(x) w_1(x) r_2(x) w_2(x) r_0(y) w_1(y)$

- Schedule in 2PL but not in TS
 - $r_2(x) w_2(x) r_1(x) w_1(x)$
- Schedule both in TS and in 2PL
 - $r_1(x) r_2(y) w_2(y) w_1(x) r_2(x) w_2(x)$

- Besides: $r_2(x) w_2(x) r_1(x) w_1(x)$ is serial but not in TS

TS and CSR

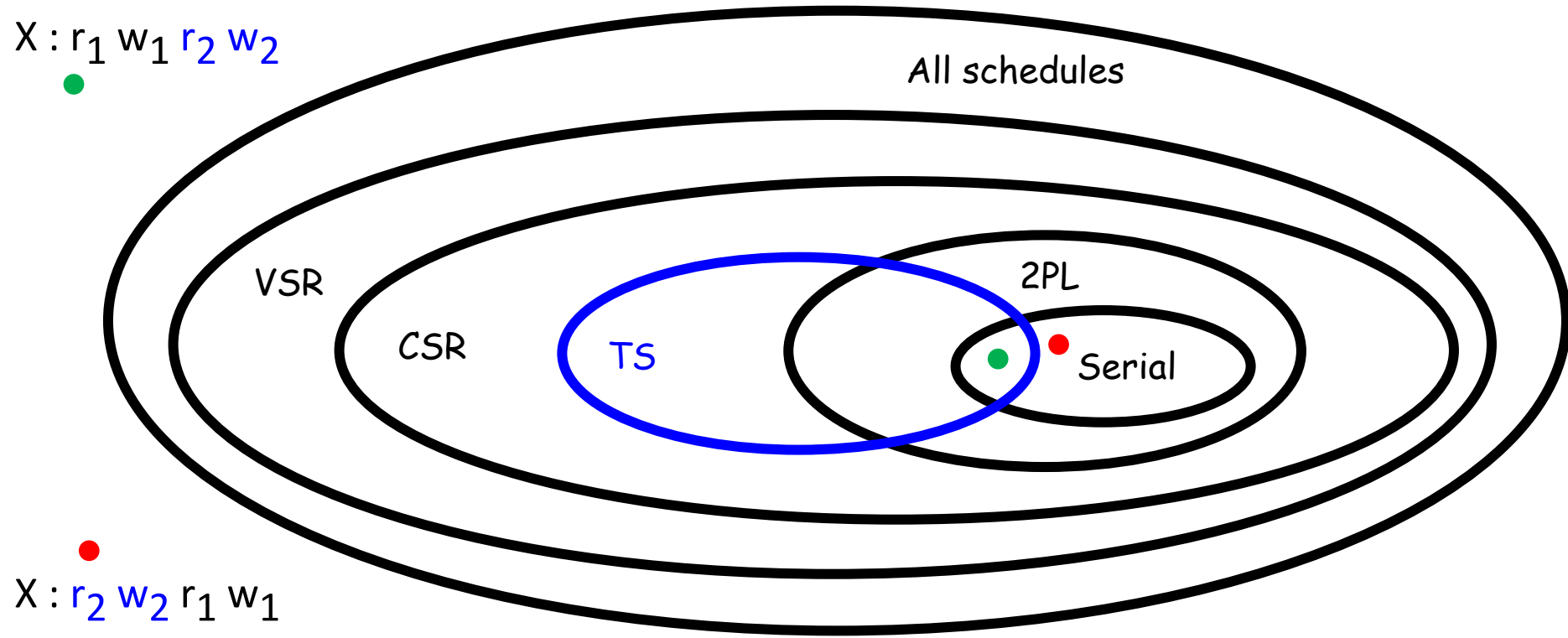
- **TS $\Rightarrow$ CSR**

- Let S be a TS schedule of T_1 and T_2
- Suppose S is not CSR, which implies that it contains a cycle between T_1 and T_2
 - S contains $op_1(x), op_2(x)$ where at least one of the op_i is a write
 - S contains also $op_2(y), op_1(y)$ where at least one of the op_i is a write
- When $op_1(y)$ arrives:
 - If $op_1(y)$ is a read, T_1 is killed by TS because it tries to read a value written by a younger transaction [$ts < WTM(y)$] [$1 < 2$] $\rightarrow$ CONTRADICTION
 - If $op_1(y)$ is a write, T_1 is killed no matter what $op_2(y)$ is because it tries to write a value already read or written by a younger transaction [$ts < R/WTM(y)$] [$1 < 2$] $\rightarrow$ CONTRADICTION

TS and dirty reads

- Basic TS-based control considers only committed transactions in the schedule, aborted transactions are not considered (commit-projection hypothesis)
- If aborts occur, dirty reads may happen
- To cope with dirty reads, a variant of basic TS must be used
 - A transaction T_i that issues a $r_i(x)$ or $w_i(x)$ such that $i > WTM(x)$ (i.e., acceptable) has its read or write operation **delayed** until the transaction $T_{WTM(x)}$ that wrote the value of x has committed or aborted
 - Similar to long duration write locks
 - But...buffering operations introduces delays

CSR, VSR, 2PL and TS



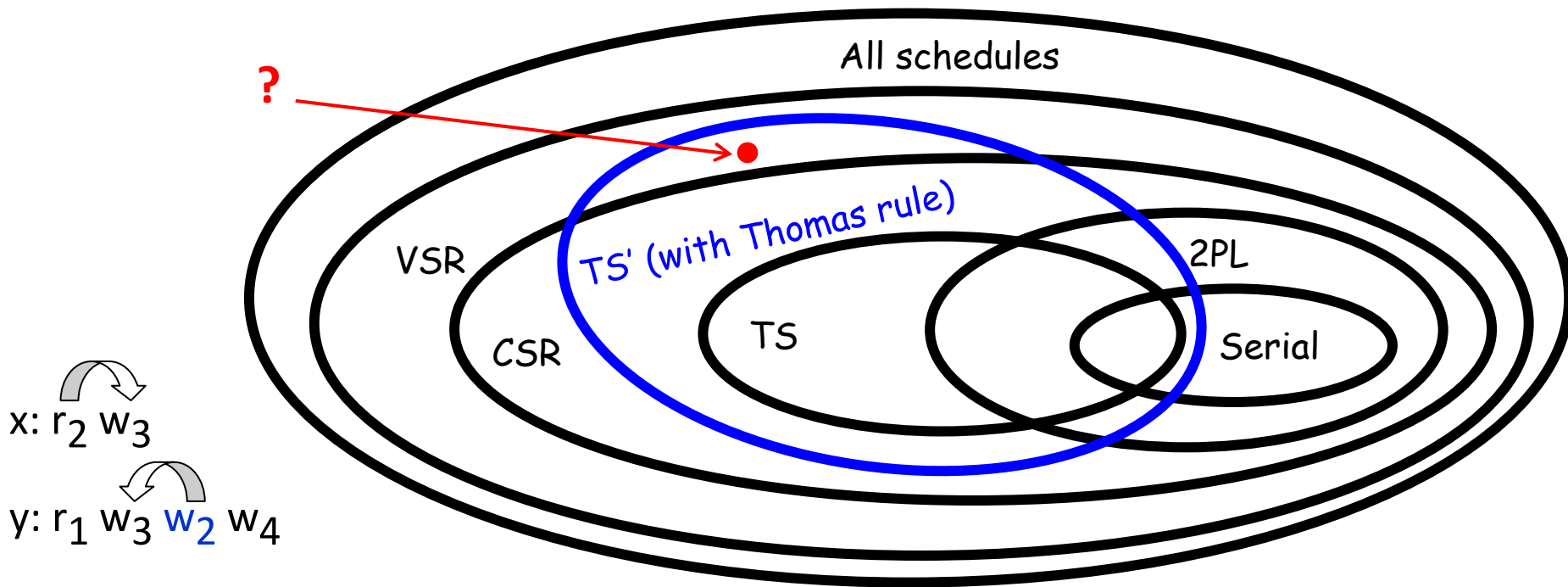
2PL vs. TS

- In 2PL transactions can be actively waiting. In TS they are killed and restarted
- The serialization order with 2PL is imposed by conflicts, while in TS it is imposed by the timestamps
- The necessity of waiting for commit of transactions causes long delays in strict 2PL
- 2PL can cause deadlocks, TS can be used to prevent deadlocks with the wound-wait and wait-die schemes explained before
- Restarting a transaction costs more than waiting: 2PL wins!
- Commercial systems implement a mix of optimistic and pessimistic concurrency control (e.g., Strict 2PL or 2PL + Multi Version TS)

Reducing kill rate: Thomas Rule

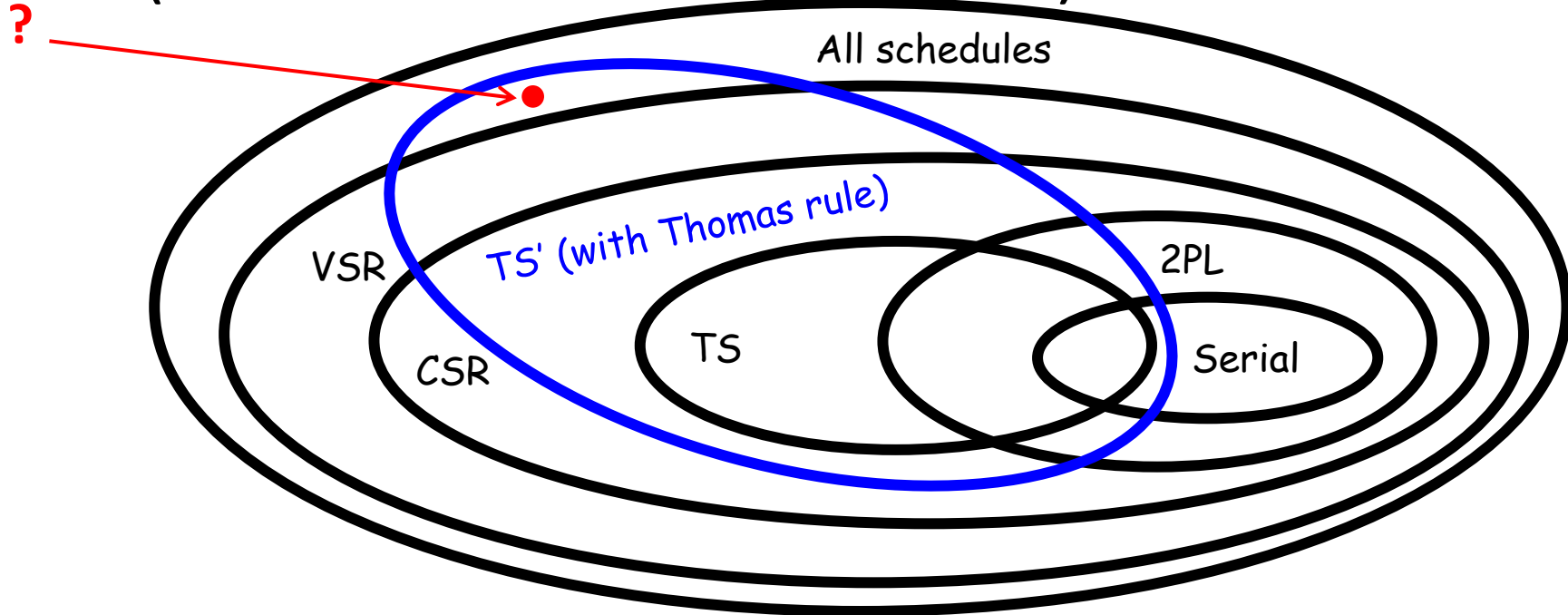
- The scheduler has two counters: $RTM(x)$ and $WTM(x)$ for each object
- The scheduler receives read/write requests tagged with timestamps:
 - $r_{ts}(x)$:
 - If $ts < WTM(x)$ the request is rejected and the transaction is killed
 - Else, access is granted and we set $RTM(x) = \max(RTM(x), ts)$
 - $w_{ts}(x)$:
 - If $ts < RTM(x)$ the request is **rejected** and the transaction is killed
 - Else, if $ts < WTM(x)$ then our write is “obsolete”: it can be **skipped**
 - Else, access is **granted** and we set $WTM(x) = ts$
- Rationale: skipping a write on an object that has already been written by a younger transaction, without killing the transaction
- Works only if the transaction issues a write without requiring a previous read on the object (so SET X = X+1 would fail)
- Does this modification affect the taxonomy of the serialization classes?

TS' (TS with Thomas Rule)



- $r_1(y) r_2(x) w_3(y) w_2(y) w_3(x) w_4(y)$ (this is an arrival sequence...)
 $w_2(y)$ is an obsolete write and thus skipped.
 - This arrival sequence is accepted by Thomas Rule, but we are changing the semantics, since not all of it is executed

TS' (TS with Thomas Rule)



- $w_2(x) \text{ } w_1(x) \text{ } r_2(x)$
 - $w_1(x)$ skipped by Thomas Rule
- Schedule not in VSR
 - T_1, T_2 different final write relation
 - T_2, T_1 different reads-from relation for $r_2(x)$

Multiversion Concurrency Control

- Idea: writes generate new versions, reads access the “right” version
- Writes generate new copies, each one with a new WTM. Each object x always has $N \geq 1$ active versions
 - The i -th version of x has its write timestamp $WTM_i(x)$
- There is a unique global $RTM(x)$
- Old versions are discarded when there are no transactions that need their values

1st version (used in theory): TS-Multi allowing unordered writes

- Mechanism:

- $r_{ts}(x)$ is always accepted. A copy x_k is selected for reading such that:
 - If $ts \geq WTM_N(x)$, then $k = N$
 - Else take k such that $WTM_k(x) \leq ts < WTM_{k+1}(x)$
- $w_{ts}(x)$:
 - If $ts < RTM(x)$ the request is **rejected**
 - Else a new version is **created** for timestamp ts (N is incremented)
 - $WTM_1(x), \dots, WTM_N(x)$ are the new versions, kept sorted from oldest to youngest
 - NB: this version shows what can be done in theory but is not the one used in the exercises

Example in theory:

TS-Multi allowing unordered writes

Assume $RTM(x) = 7$, $N=1$, $WTM_1(x) = 4$

<i>Request</i>	<i>Response</i>	<i>New Value</i>
$r_6(x)$	ok	
$r_8(x)$	ok	$RTM(x) = 8$
$r_9(x)$	ok	$RTM(x) = 9$
$w_8(x)$	no	T_8 killed
$w_{11}(x)$	ok	$WTM_2(x) = 11$, $N=2$
$r_{10}(x)$	ok on $x_{(1)}$	$RTM(x) = 10$ (no longer killed)
$r_{12}(x)$	ok on $x_{(2)}$	$RTM(x) = 12$
$w_{14}(x)$	ok	$WTM_3(x) = 14$, $N=3$
$w_{13}(x)$	ok	$WTM_1(x)=4$, $WTM_2(x)=11$, $WTM_3(x)=13$, $WTM_4(x)=14$, $N=4$ (no need to kill)

2nd version (used in practice): TS-Multi under Snapshot Isolation

- Mechanism:

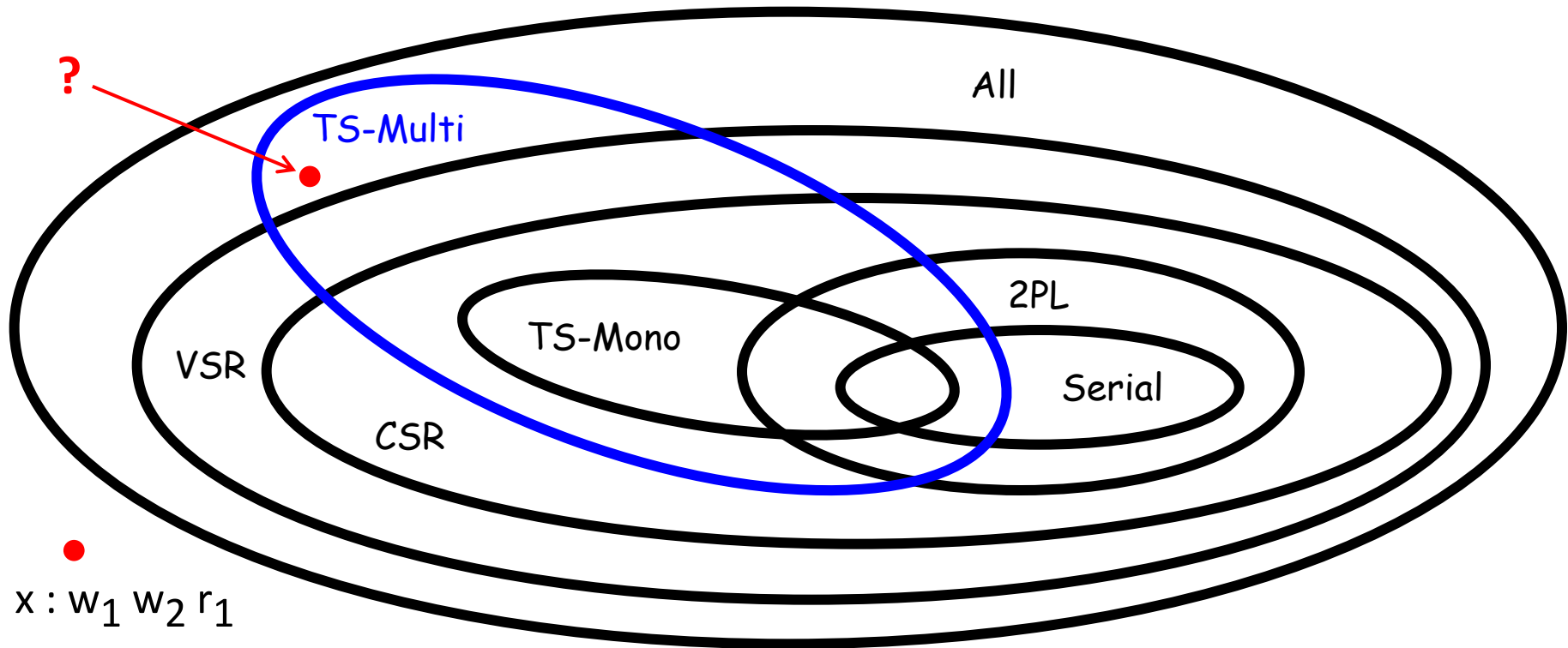
- $r_{ts}(x)$ is always accepted. A copy x_k is selected for reading such that:
 - If $ts \geq WTM_N(x)$, then $k = N$
 - Else take k such that $WTM_k(x) \leq ts < WTM_{k+1}(x)$
- $w_{ts}(x)$:
 - If $ts < RTM(x)$ or $ts < WTM_N(x)$ the request is rejected
 - Else a new version is created for timestamp ts (N is incremented)
 - $WTM_1(x), \dots, WTM_N(x)$ are the new versions, kept sorted from oldest to youngest
 - NB: this version is used in real systems based, e.g., on *snapshot isolation* (see later) and in the exercises

Example in practice (for the exam): TS-Multi under Snapshot Isolation

Assume $RTM(x) = 7$, $N=1$, $WTM_1(x) = 4$

<i>Request</i>	<i>Response</i>	<i>New Value</i>
$r_6(x)$	ok	
$r_8(x)$	ok	$RTM(x) = 8$
$r_9(x)$	ok	$RTM(x) = 9$
$w_8(x)$	no	T_8 killed
$w_{11}(x)$	ok	$WTM_2(x) = 11$, $N=2$
$r_{10}(x)$	ok on $x_{(1)}$	$RTM(x) = 10$ (no longer killed)
$r_{12}(x)$	ok on $x_{(2)}$	$RTM(x) = 12$
$w_{14}(x)$	ok	$WTM_3(x) = 14$, $N=3$
$w_{13}(x)$	no	but in the exercises, T_{13} is killed

CSR, VSR, 2PL, TS-Mono, TS-Multi



- Versions: $x_{(0)}$ (original value), $x_{(1)}$, $x_{(2)}$
- r_1 reads $x_{(1)}$ [$WTM_k(x) \leq ts < WTM_{k+1}(x)$]
- Schedule not in VSR
 - T_1, T_2 different reads-from relation for $r_1(x)$
 - T_2, T_1 different final write relation

Snapshot Isolation (SI)

- The realization of TS-Multi gives the opportunity to introduce into DBMSs another isolation level, **SNAPSHOT ISOLATION**
- In this level, no RTM is used on the objects, only WTM
- Every transaction reads the version consistent with its timestamp (i.e., the version that existed when the transaction started a.k.a. **snapshot**), and defers writes to the end
- If the scheduler detects that the writes of a transaction conflict with writes of other concurrent transactions after the snapshot timestamp, it aborts
 - It is yet another case of optimistic concurrency control

Anomalies in Snapshot Isolation

- Snapshot isolation does not guarantee serializability
 - T_1 : update Balls set Color=White where Color=Black
 - T_2 : update Balls set Color=Black where Color=White
- Serializable executions of T_1 and T_2 will produce a final configuration with balls that are either all white or all black
- An execution under Snapshot Isolation in which the two transactions start with the same snapshot will just swap the two colors
- This anomaly is called **write skew**

Is VSR the ultimate class?

- The schedule $w_1(x) w_2(x) r_1(x)$ is outside VSR but was accepted by TS-Multi: why?
 - With TS-Multi, we are changing the semantics of execution!
- Are all good schedules in VSR?
- We can prove that, for all schedules outside VSR, there is an execution that leads to a final state different from the one that would be reached with any serial schedule of the same operations
 - Yet, this does not mean that the schedule may lead to anomalies
 - Take, e.g., $r_1(x) w_2(x) w_1(x)$: not in VSR but no anomalies... as we have defined them!
- What are all the anomalies? Are our (7) anomalies the only relevant ones?

Concurrency control in some commercial DBMS

- ORACLE
- https://docs.oracle.com/cd/B19306_01/server.102/b14220/consist.htm
- MYSQL
- <https://dev.mysql.com/doc/refman/8.0/en/locking-issues.html>
- IBM DB2
- https://www.ibm.com/support/knowledgecenter/en/SSEPGG_9.7.0/com.ibm.db2.luw.admin.perf.doc/doc/c0054923.html
- MONGODB
- <https://docs.mongodb.com/manual/faq/concurrency/>